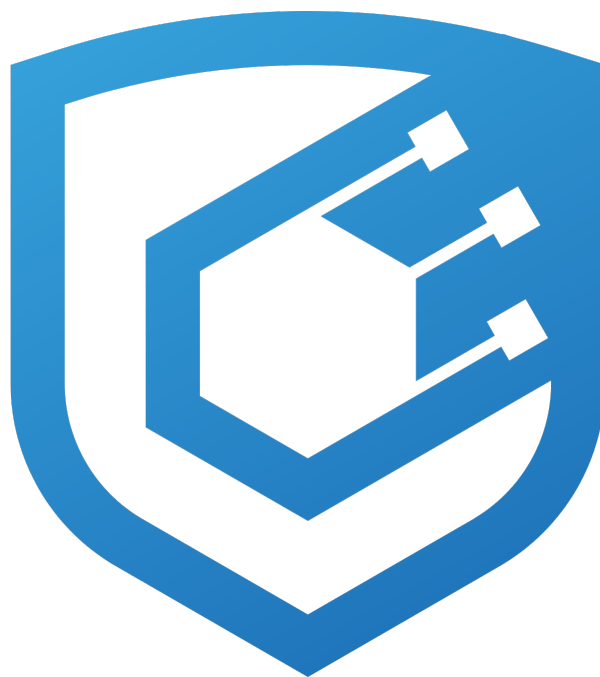




Syntetika CCIP-CCT Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Immeas](#)

[MrPotatoMagic](#)

May 2, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
4.1	Protocol Overview	3
4.2	Core Components	3
4.3	Cross-Chain Architecture (CCIP-CCT)	4
4.4	Access Control & Upgradeability	4
5	Audit Scope	4
6	Executive Summary	5
6.1	Centralization Risks	5
6.2	Defensive Improvements	5
6.3	Scope change	6
7	Findings	9
7.1	High Risk	9
7.1.1	Minter::realizeLosses is dead by default: reverts on missing vault to Minter allowance	9
7.2	Medium Risk	13
7.2.1	StakingVault::claimWithdraw burns surplus on post-redeem losses, double-penalizing remaining stakers	13
7.2.2	HilToken::_update inverted waiting-period check + permissionless HilToken::requestTransfer enables single-tx force-burn of any holder's balance	17
7.2.3	Distributor::distributeYield permissionless + no yieldAmount validation - grieving burns rate-limit slot	18
7.3	Low Risk	19
7.3.1	StakingVault::withdraw, redeem silently ignore ERC-4626 receiver and owner parameters - causes fund loss for aggregators	19
7.3.2	Minter::ownerMint inverted waiting-period check enables single-tx unbacked mint; chained with self-whitelist drains all Minter baseAsset	20
7.3.3	Direct grantRole, revokeRole bypasses the 2-step role mechanism across Minter, HilToken, StakingVault, Distributor	20
7.3.4	Minter::totalDeposits accounting drifts from both reserves and synthetic supply across every privileged path	21
7.3.5	Minter::mint docstring vs code: recipient is never checked against whitelist	22
7.3.6	Events / event-argument inconsistencies across contracts	23
7.3.7	HilToken admin blacklisting TokensHolder or StakingVault in its blacklist freezes every vault exit - cross-contract blacklist DoS by a single HilToken admin key	25
7.3.8	Missing runtime input validation on admin setters and creator-role functions	26
7.3.9	BlacklistableUpgradeable uses single-step OwnableUpgradeable — instant ownership transfer bypasses all owner()-gated invariants	26
7.3.10	StakingVault::redeem, withdraw missing zero-output guard in no-cooldown path	27
7.3.11	Fully-vested yield becomes unvested on vesting period update	28
7.3.12	Minter::distributeYield missing whenNotPaused	29
7.3.13	BlacklistableUpgradeable::blacklist(address(0)) bricks all mint and burn paths across HilToken and StakingVault	30
7.4	Informational	31
7.4.1	StakingVault::_withdraw last-user branch reverts with ERC20InvalidReceiver when owner is renounced to zero - DoS of last-user exit during vesting window	31
7.4.2	TokensHolder::withdraw + Distributor::rescueERC20 use raw transfer without SafeERC20	31

7.4.3	StakingVault::setEarlyExitFeeRecipient defaults to address(this) and permits self-target - fees silently boost share price	32
7.4.4	StakingVault::_deposit with minAssetsAmount > DEAD_SHARES bricks vault seeding . . .	32
7.4.5	StakingVault::setDistributor and Minter::setDistributor are independent - rotating one without the other silently bricks the yield pipeline	32
7.4.6	CUSTODIAN_ROLE is granted/revoked but never gates any function	32
7.4.7	Distributor::rescueERC20 uses string error messages inconsistent with project style and ignores transfer return value	33
7.4.8	Missing NatSpec on public-facing functions with non-obvious behaviour	33
7.4.9	BlacklistableUpgradeable::_Blacklistable_init callable publicly during initialization .	34
7.4.10	TokensHolder::withdraw discards transfer return value and missing event	34
7.4.11	StakingVault::_withdraw resets \$.lastDistributionTimestamp to 0	34
7.4.12	Documentation mismatches: NatSpec, docstrings and comments disagree with code	34
7.4.13	Missing zero-address and zero-amount input validation across constructors, initializers and setters	35
7.4.14	Solidity compiler version and toolchain configuration drift across the repo	36
7.4.15	Cosmetic code cleanups: typo, unused imports, inconsistent types and wrong error name . .	36
7.4.16	Inconsistent access-control patterns: mixed Ownable/AccessControl and mixed internal/external role revocation	37
7.4.17	HilToken::_update and StakingVault::_update check blacklist status of address(0) . . .	38
7.5	Gas Optimization	40
7.5.1	Cache repeated storage reads on success path across multiple setters	40
7.5.2	Use calldata instead of memory for external function string parameters	40

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Syntetika is an innovative "real yield" solution that bridges traditional Bitcoin with DeFi and sophisticated trading strategies, previously [audited](#) by Cyfrin.

4.1 Protocol Overview

Syntetika issues compliant synthetic tokens, **HiIBTC** (18 decimals, pegged to Bitcoin) and **HiIUSD** (6 decimals, pegged to USD), that represent user deposits of real collateral assets. These synthetic tokens accrue yield generated from off-chain Bitcoin trading strategies, with on-chain distribution managed transparently through a set of upgradeable smart contracts. The protocol is designed with institutional compliance requirements in mind: all token transfers are gated through a configurable policy engine and an on-chain KYC/AML layer, and every sensitive admin operation is protected by a two-step time-delayed pattern.

4.2 Core Components

Minter The central issuance contract. Whitelisted users deposit a baseAsset and receive an equal nominal amount of the corresponding synthetic token (HiIBTC or HiIUSD) via `mint`. The inverse, `redeem`, burns synthetic tokens and returns the baseAsset. A separate admin-only `ownerMint` path (two-step, one-day delay) allows direct issuance without collateral, intended for protocol-level accounting adjustments. The Minter holds custody of all collateral and periodically routes it to a designated custodian via `transferToCustody`. Yield is injected back into the system through `distributeYield`, which mints additional synthetic tokens credited to the StakingVault.

HiIToken / HiIBTC / HiIUSD ERC-20 compliant synthetic tokens built on a shared HiIToken base. Access to `mint` and `burn` is gated by `MINTER_ROLE` (held by the Minter contract and, in multi-chain deployments, by Chainlink CCT token pool contracts). Token transfers are subject to a compliance overlay: non-whitelisted addresses go through a Chainlink `PolicyEngine` check, and blacklisted addresses are blocked at every transfer, mint, and burn. The owner retains a privileged force-transfer path via a two-step `requestTransfer` / `_update` mechanism for regulatory intervention.

StakingVault An ERC-4626 vault where users deposit HiBTC or HiUSD to earn yield. Shares represent proportional ownership of the vault's growing asset base. Yield is credited by the Distributor and vests linearly over a configurable vesting period to smooth the per-share exchange rate. Withdrawals follow a two-phase flow: users call `redeem/withdraw` to start a cooldown (up to 90 days), after which assets held in escrow by `TokensHolder` can be claimed. An optional early-exit mechanism lets users claim before the cooldown ends at the cost of a configurable fee (capped at 10%) paid to a designated recipient. Repeated cooldown requests will reset the cooldown timer. Losses during a cooldown will affect the amount of tokens received during claim.

Distributor A permissionless yield-relay contract that sits between off-chain yield sources and the StakingVault. Any caller may trigger `distributeYield`, which reads the current yield amount from the StakingVault and instructs the Minter to mint the corresponding synthetic tokens directly into the vault. Successive distributions are rate-limited by a configurable `timeLock` (minimum interval between calls) to prevent rapid compounding manipulation.

CompliantDepositRegistry Manages a registry of pre-approved deposit addresses for users bridging from traditional finance. Addresses are submitted in batches that enter a challenge period before being finalized; a designated canceler can invalidate a pending batch if suspicious addresses are detected. Once finalized, registered addresses can be claimed by users to establish a compliant on-chain presence.

ComplianceCheckerUpgradeable Implements SBT (Soulbound Token) based KYC/AML verification. An address is deemed compliant if it holds the SBTs required by at least one configured compliance option. The compliance options are managed by a dedicated admin role and feed into the `PolicyEngine` used by the Minter's whitelist layer.

Supporting Helpers

- **Whitelist / PolicyEngine integration:** The `Whitelist` base contract (inherited by Minter) routes non-whitelisted callers through a Chainlink `PolicyEngine` for on-chain compliance checks before any state change is allowed.
- **BlacklistableUpgradeable:** A shared blacklist implementation used by both `HiToken` and `StakingVault`. Blacklisted addresses cannot send, receive, or burn tokens.
- **TokensHolder:** A simple escrow contract that holds users' underlying assets during the `StakingVault` cooldown period until they are claimed.

4.3 Cross-Chain Architecture (CCIP-CCT)

HiBTC and HiUSD are deployed as Chainlink Cross-Chain Tokens (CCT) using the `BurnAndMint` pool model. On the origin chain the Minter holds and manages collateral; on remote chains Chainlink CCIP `BurnMintTokenPool` contracts hold `MINTER_ROLE` on the local `HiToken` deployment, enabling native mint/burn without liquidity fragmentation. This allows institutional users to hold and transfer synthetic Syntetika tokens across multiple EVM networks while collateral remains consolidated on the origin chain.

The `StakingVault` integration uses a `LockRelease` CCIP-CCT model: vault shares are locked on the source chain and released on the destination chain rather than burned and reminted, preserving the share accounting and yield-accrual state that lives exclusively on the origin chain.

4.4 Access Control & Upgradeability

The issuance contracts (Minter, HiBTC, HiUSD, StakingVault, Distributor) are deployed as UUPS proxies. `CompliantDepositRegistry` and `TokensHolder` are non-upgradeable. Sensitive operations — role rotations, upgrade authorizations, owner mints — follow a two-step request/confirm pattern enforced by a configurable waiting period (default: one day). Roles are managed through OpenZeppelin `AccessControl`. Key roles include `DEFAULT_ADMIN_ROLE`, `OPERATOR_ROLE`, `CUSTODIAN_ROLE`, `PAUSER_ROLE`, `DISTRIBUTOR_ROLE`, `MINTER_ROLE`, and `BLACKLISTER_ROLE`, with documented intent that the admin roles are held by a multi-signature wallet.

5 Audit Scope

The focus of this audit were changes to support Chainlink's [CCT Standard](#) and some security-focused design changes. The audit scope was limited to:

```
issuance/src/helpers/TokensHolder.sol
issuance/src/helpers/BlacklistableUpgradeable.sol
issuance/src/helpers/Whitelist.sol
issuance/src/minter/Minter.sol
issuance/src/token/HilBTC.sol
issuance/src/token/HilToken.sol
issuance/src/token/HilUSD.sol
issuance/src/vault/Distributor.sol
issuance/src/vault/StakingVault.sol
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the [Syntetika CCIP-CCT](#) code provided by [Syntetika](#). The findings consist of 1 High, 3 Medium & 13 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

The protocol is centralized by design. The following administrative powers are documented design, not bugs:

- PAUSER_ROLE can pause and unpause the Minter, halting mint, redeem, transferToCustody, and realizeLosses.
- The admin can upgrade any UUPS contract (Minter, HilToken, StakingVault, Distributor).
- The admin can rescue arbitrary ERC-20 tokens from the Distributor via Distributor::rescueERC20.
- The blacklister (initialized to the admin) can blacklist or unblacklist any address via BlacklistableUpgradeable::blacklist, unBlacklist, including protocol-internal addresses, which in HilToken can freeze vault exit paths.
- The admin can grantRole/revokeRole on any of the nine defined roles (DEFAULT_ADMIN, CUSTODIAN, OPERATOR, PAUSER, DISTRIBUTOR, MINTER, DEPOSIT_ADDRESS_CREATOR, CANCELER, COMPLIANCE_ADMIN).
- The admin can transfer or renounce Ownable ownership on StakingVault, HilToken via single-step inherited calls (renouncing to the zero address is reported separately as a finding).
- The admin controls the IFeed oracle via Distributor::updateFeed; combined with the permissionless distributeYield this creates a peg-break / grieving surface (reported separately).
- The admin controls fee and cooldown parameters within documented ranges: maxEarlyExitFeeBps, earlyExitFeeRecipient, cooldownDuration, minAssetsAmount.
- OPERATOR_ROLE can drain the entire Minter baseAsset balance to the custodian via Minter::transferToCustody without any on-chain cap.

Users must place substantial trust in the admins. The positive counterpart is that if a serious bug is discovered after deployment, admins can upgrade the affected contracts and resolve the issue.

6.2 Defensive Improvements

The following gaps are defense-in-depth weaknesses rather than exploitable bugs. Individually each is Info-to-Low; collectively they expose the protocol to single-transaction exploit paths under admin-key compromise or operator misbehaviour. Addressing them would materially harden the protocol's posture.

- HilToken::mint **has no per-minter or per-epoch cap** (HilToken.sol:149-151). Once CCT pools are granted MINTER_ROLE, any minter bug scales 1:1 to unbounded supply. Add a per-minter daily cap gated by DEFAULT_ADMIN_ROLE.

- **CompliantDepositRegistry is non-upgradeable** while every other issuance contract is UUPS, and DEPOSIT_ADDRESS_CREATOR_ROLE is not granted in the constructor (CompliantDepositRegistry.sol:14,56-57). A registry bug would require full migration with split-state risk; a forgotten role grant DoSes the registry. Migrate to UUPS and grant the creator role in the constructor.
- **StakingVault, HilToken have no pause mechanism.** Incidents at the vault or token layer can only be addressed via per-address blacklist or 1-day emergency upgrade. Add PausableUpgradeable to both and gate critical user entry points with whenNotPaused.
- **StakingVault first-depositor donation-inflation moat is thin.** DEAD_SHARES = 1000 on 8-decimal HilBTC is roughly \$1 at current BTC prices (StakingVault.sol:57,198). The _checkOwner first-deposit gate blocks the classic front-run, but small subsequent deposits remain exposed to rounding griefing. Require the owner's initial seed to be economically significant (at least \$100).
- **No separate UPGRADE_ADMIN_ROLE or pause/unpause split.** DEFAULT_ADMIN_ROLE gates both parameter setters and _authorizeUpgrade across Minter, HilToken, StakingVault, Distributor, so a leak of the operational key also owns the proxy. Move upgrade authority to a distinct UPGRADE_ADMIN_ROLE held by a multisig/timelock with a longer minimum delay, and split PAUSER_ROLE so pausing can move faster than unpausing.
- **StakingVault admin setters are single-step** (StakingVault.sol:138-181,414-436) while only upgrades are 2-step. setCooldownDuration, setEarlyExitEnabled, setMaxEarlyExitFeeBps, setEarlyExitFeeRecipient, setMinAssetsAmount, setDistributor all apply retroactively to in-flight cooldowns. Wrap each in a 2-step request/execute pattern (waiting period >= cooldownDuration) and snapshot the relevant parameters into each user's UserCooldown at redeem time.
- **Distributor has no 2-step pattern at all** (Distributor.sol:86-119,177-179). _authorizeUpgrade is empty, requestUpgrade does not exist, and updateFeed, updateMinter, updateTimeLock are single-step. Because Distributor retains DISTRIBUTOR_ROLE on Minter across proxy upgrades, a malicious upgrade is a single-block drain path. Mirror Minter's request/execute pattern for upgrades and the three setters; add MIN_TIME_LOCK/MAX_TIME_LOCK bounds.
- **Minter::attachPolicyEngine is onlyOwner and single-step** via inherited PolicyProtected (Minter.sol:151, Whitelist.sol:37). A compromised owner key can swap in an attacker-controlled PolicyEngine in one transaction, bypassing onlyWhitelisted on mint and redeem. Override attachPolicyEngine in Whitelist or Minter with a 2-step request/execute pattern matching the other Minter setters.

6.3 Scope change

Initially the two contract deposit-registry/contracts/ComplianceCheckerUpgradeable.sol and deposit-registry/contracts/CompliantDepositRegistry.sol were part of the scope but later removed. Any findings related to these contracts have been removed from the report.

Summary

Project Name	Syntetika CCIP-CCT
Repository	monorepo
Commit	16a07be4bb3a...
Fix Commit	34eb00140d37...
Audit Timeline	Apr 20th - Apr 24th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	1
Medium Risk	3
Low Risk	13
Informational	17
Gas Optimizations	2
Total Issues	36

Summary of Findings

[H-1] <code>Minter::realizeLosses</code> is dead by default: reverts on missing vault to Minter allowance	Resolved
[M-1] <code>StakingVault::claimWithdraw</code> burns surplus on post-redeem losses, double-penalizing remaining stakers	Resolved
[M-2] <code>HilToken::_update</code> inverted waiting-period check + permissionless <code>HilToken::requestTransfer</code> enables single-tx force-burn of any holder's balance	Resolved
[M-3] <code>Distributor::distributeYield</code> permissionless + no <code>yieldAmount</code> validation - grieving burns rate-limit slot	Resolved
[L-01] <code>StakingVault::withdraw</code> , <code>redeem</code> silently ignore ERC-4626 receiver and owner parameters - causes fund loss for aggregators	Resolved
[L-02] <code>Minter::ownerMint</code> inverted waiting-period check enables single-tx unbacked mint; chained with self-whitelist drains all Minter baseAsset	Resolved
[L-03] Direct <code>grantRole</code> , <code>revokeRole</code> bypasses the 2-step role mechanism across Minter, HilToken, StakingVault, Distributor	Resolved
[L-04] <code>Minter::totalDeposits</code> accounting drifts from both reserves and synthetic supply across every privileged path	Resolved
[L-05] <code>Minter::mint</code> docstring vs code: recipient is never checked against whitelist	Resolved
[L-06] Events / event-argument inconsistencies across contracts	Resolved
[L-07] HilToken admin blacklisting TokensHolder or StakingVault in its blacklist freezes every vault exit - cross-contract blacklist DoS by a single HilToken admin key	Resolved
[L-08] Missing runtime input validation on admin setters and creator-role functions	Resolved
[L-09] BlacklistableUpgradeable uses single-step OwnableUpgradeable — instant ownership transfer bypasses all <code>owner()</code> -gated invariants	Resolved
[L-10] <code>StakingVault::redeem</code> , <code>withdraw</code> missing zero-output guard in no-cooldown path	Resolved
[L-11] Fully-vested yield becomes unvested on vesting period update	Resolved

[L-12] Minter::distributeYield missing whenNotPaused	Resolved
[L-13] BlacklistableUpgradeable::blacklist(address(0)) bricks all mint and burn paths across HilToken and StakingVault	Resolved
[I-01] StakingVault::_withdraw last-user branch reverts with ERC20InvalidReceiver when owner is renounced to zero - DoS of last-user exit during vesting window	Resolved
[I-02] TokensHolder::withdraw + Distributor::rescueERC20 use raw transfer without SafeERC20	Resolved
[I-03] StakingVault::setEarlyExitFeeRecipient defaults to address(this) and permits self-target - fees silently boost share price	Resolved
[I-04] StakingVault::_deposit with minAssetsAmount > DEAD_SHARES bricks vault seeding	Resolved
[I-05] StakingVault::setDistributor and Minter::setDistributor are independent - rotating one without the other silently bricks the yield pipeline	Resolved
[I-06] CUSTODIAN_ROLE is granted/revoked but never gates any function	Resolved
[I-07] Distributor::rescueERC20 uses string error messages inconsistent with project style and ignores transfer return value	Resolved
[I-08] Missing NatSpec on public-facing functions with non-obvious behaviour	Resolved
[I-09] BlacklistableUpgradeable::_Blacklistable_init callable publicly during initialization	Resolved
[I-10] TokensHolder::withdraw discards transfer return value and missing event	Resolved
[I-11] StakingVault::_withdraw resets \$.lastDistributionTimestamp to 0	Resolved
[I-12] Documentation mismatches: NatSpec, docstrings and comments disagree with code	Resolved
[I-13] Missing zero-address and zero-amount input validation across constructors, initializers and setters	Resolved
[I-14] Solidity compiler version and toolchain configuration drift across the repo	Resolved
[I-15] Cosmetic code cleanups: typo, unused imports, inconsistent types and wrong error name	Resolved
[I-16] Inconsistent access-control patterns: mixed Ownable/AccessControl and mixed internal/external role revocation	Resolved
[I-17] HilToken::_update and StakingVault::_update check blacklist status of address(0)	Resolved
[G-1] Cache repeated storage reads on success path across multiple setters	Resolved
[G-2] Use calldata instead of memory for external function string parameters	Resolved

7 Findings

7.1 High Risk

7.1.1 Minter::realizeLosses is dead by default: reverts on missing vault to Minter allowance

Description: Minter::realizeLosses calls hilSyntheticToken.burnFrom(address(vault), lossesAmount). In HilToken::burnFrom the allowance check is skipped only when spender == from OR spender == owner(). Here spender == Minter, from == StakingVault, and owner() == initialAdmin (distinct from the Minter proxy). Thus _spendAllowance(vault, Minter, amount) executes - the StakingVault never approves the Minter. No approve call from StakingVault to Minter exists in any deploy script or in StakingVault::initialize, and no function on StakingVault (or elsewhere reachable by admin) grants this allowance. Default allowance = 0 -> ERC20InsufficientAllowance revert for any non-zero loss.

This is distinct from the client-acknowledged "realizeLosses front-run DoS" known issue. Front-running implies the function normally works but can be perturbed; this finding shows the function can NEVER work in a standard deployment.

Source: issuance/src/minter/Minter.sol:431-440.

Impact: The protocol's only loss-realization path is unusable. The vault can only grow (yield minted) and never shrink to reflect custody losses, producing permanent over-collateralization drift and mispricing sharePrice during real losses. Admin's only alternative is a full protocol upgrade - exactly the destructive control the 2-step admin was meant to prevent.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "forge-std/Test.sol";
import {Minter} from "../../src/minter/Minter.sol";
import {HilBTC} from "../../src/token/HilBTC.sol";
import {HilToken} from "../../src/token/HilToken.sol";
import {StakingVault} from "../../src/vault/StakingVault.sol";
import {Distributor} from "../../src/vault/Distributor.sol";
import {MockFeed} from "../../src/vault/MockFeed.sol";
import {IStakingVault} from "../../src/interfaces/vault/IStakingVault.sol";
import {IFeed} from "../../src/interfaces/chainlink/IFeed.sol";
import {IMinter} from "../../src/interfaces/minter/IMinter.sol";
import {MockERC20} from "../../mocks/MockERC20.sol";
import {MockPolicyEngine} from "../../src/kyc/MockPolicyEngine.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {
    ERC1967Proxy
} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

contract H02_RealizeLossesDead is Test {
    HilBTC hilBTC;
    MockERC20 baseAsset;
    StakingVault vault;
    Minter minter;
    Distributor distributor;
    MockFeed feed;

    address admin = address(0x1111);
    address custodian = address(0x2222);
    address operator = address(0x3333);
    address pauser = address(0x4444);
    address user1 = address(0x5555);

    function setUp() public {
        vm.warp(1770880586);
```

```

vm.startPrank(admin);

baseAsset = new MockERC20();

HilBTC hilImpl = new HilBTC();
bytes memory hilInit = abi.encodeWithSelector(
    HilToken.initialize.selector,
    "HilBTC",
    "hilBTC",
    admin,
    address(0)
);
ERC1967Proxy hilProxy = new ERC1967Proxy(address(hilImpl), hilInit);
hilBTC = HilBTC(address(hilProxy));

StakingVault vaultImpl = new StakingVault();
bytes memory vaultInit = abi.encodeWithSelector(
    StakingVault.initialize.selector,
    IERC20(address(hilBTC)),
    "Staked HilBTC",
    "sHilBTC",
    admin,
    1_000,
    address(0)
);
ERC1967Proxy vaultProxy = new ERC1967Proxy(
    address(vaultImpl),
    vaultInit
);
vault = StakingVault(address(vaultProxy));

Minter minterImpl = new Minter();
MockPolicyEngine policyEngine = new MockPolicyEngine();
bytes memory minterInit = abi.encodeWithSelector(
    Minter.initialize.selector,
    admin,
    address(baseAsset),
    address(hilBTC),
    custodian,
    operator,
    pauser,
    admin,
    address(policyEngine),
    IStakingVault(address(vault))
);
ERC1967Proxy minterProxy = new ERC1967Proxy(
    address(minterImpl),
    minterInit
);
minter = Minter(address(minterProxy));

hilBTC.requestMinterChange(address(minter));
hilBTC.setMinter();

Distributor distrImpl = new Distributor();
feed = new MockFeed();
bytes memory distrInit = abi.encodeWithSelector(
    Distributor.initialize.selector,
    admin,
    IFeed(address(feed)),
    IMinter(address(minter)),
    8 hours
);

```

```

ERC1967Proxy distrProxy = new ERC1967Proxy(
    address(distrImpl),
    distrInit
);
distributor = Distributor(address(distrProxy));

vault.setDistributor(address(minter));

minter.requestDistributorChange(address(distributor));
vm.warp(block.timestamp + 2 days);
minter.setDistributor();

vault.setCooldownDuration(0);

minter.whitelistAddress(admin, true);
minter.whitelistAddress(user1, true);

vm.stopPrank();

baseAsset.mint(admin, 1_000e8);
baseAsset.mint(user1, 1_000e8);

vm.prank(admin);
baseAsset.approve(address(minter), type(uint256).max);

vm.prank(user1);
baseAsset.approve(address(minter), type(uint256).max);

vm.startPrank(admin);
minter.mint(admin, 500e8);
hilBTC.approve(address(vault), type(uint256).max);
vault.stake(100e8, "");
vm.stopPrank();

vm.startPrank(user1);
minter.mint(user1, 500e8);
hilBTC.approve(address(vault), type(uint256).max);
vault.stake(200e8, "");
vm.stopPrank();

vm.prank(admin);
feed.setYieldAmount(50e8);
distributor.distributeYield();

assertGt(hilBTC.balanceOf(address(vault)), 0, "vault has no hilBTC");
}

function test_PoC_realizeLossesRevertsDueToMissingAllowance() public {
    uint256 lossesAmount = 25e8;

    assertEq(
        hilBTC.allowance(address(vault), address(minter)),
        0,
        "vault unexpectedly pre-approved minter"
    );

    vm.prank(admin);
    vm.expectRevert();
    distributor.realizeLosses(lossesAmount);

    vm.startPrank(admin);
    minter.requestDistributorChange(admin);
    vm.warp(block.timestamp + 2 days);

```

```

    minter.setDistributor();
    vm.expectRevert();
    minter.realizeLosses(lossesAmount);
    vm.stopPrank();
}

function test_PoC_realizeLossesWorksAfterAllowance() public {
    uint256 lossesAmount = 25e8;
    uint256 vaultBalBefore = hilBTC.balanceOf(address(vault));

    vm.prank(address(vault));
    hilBTC.approve(address(minter), type(uint256).max);

    vm.prank(admin);
    distributor.realizeLosses(lossesAmount);

    uint256 vaultBalAfter = hilBTC.balanceOf(address(vault));
    assertEq(
        vaultBalBefore - vaultBalAfter,
        lossesAmount,
        "vault hilBTC not burned by expected amount"
    );
}
}

```

Both test cases pass: the first confirms `ERC20InsufficientAllowance(Minter, 0, lossesAmount)` on both the `Distributor::realizeLosses -> Minter::realizeLosses` path and the direct `Minter::realizeLosses` path; the second shows the function works only after simulating a vault-side approve, which no admin function currently exposes (requires a proxy upgrade).

Recommended Mitigation: Either:

- Have `StakingVault` grant `Minter` `forceApprove(minter, type(uint256).max)` during initialization; or
- Change `realizeLosses` to have the vault burn its own balance via an `onlyMinter` function on the vault; or
- In `HilToken::burnFrom`, skip allowance when `spender == $.minter` (treat the minter role as privileged for `burnFrom` like owner).

Syntetika: Fixed in commit [9cdb8d7](#)

Cyfrin: Verified. Allowance skipped when spender is minter

7.2 Medium Risk

7.2.1 StakingVault::claimWithdraw burns surplus on post-redeem losses, double-penalizing remaining stakers

Description: When a user redeems, their shares are burned immediately and `_initialAssets` are escrowed to `TokensHolder` at the redeem-time exchange rate. At claim time:

```
uint256 previewedAssets = previewRedeem(userCooldown.underlyingShares);
uint256 _initialAssets = userCooldown.underlyingAmount;
uint256 assets = _initialAssets > previewedAssets ? previewedAssets : _initialAssets;
...
if (assets < _initialAssets) {
    $.tokensHolder.withdraw(address(this), _initialAssets - assets);
    IHilToken(asset()).burn(_initialAssets - assets);
}
```

Two compounding issues:

(a) **Asymmetric floor:** the user is always paid `min(_initialAssets, previewedAssets)`. If share price rises during cooldown (yield vested), they get `_initialAssets` - no upside. If share price falls (`realizeLosses` fires), they get `previewedAssets` - full downside. Cooldown holders absorb 100% of loss risk but 0% of yield upside.

(b) **Surplus burn misdirects value:** the difference `_initialAssets - assets` of `HilBTC` is burned directly, bypassing the Minter's redeem path. Because `HilBTC` is minted 1:1 against a base asset (BTC/wBTC) held in the Minter, burning `HilBTC` outside the Minter leaves the corresponding base asset with no claimant, though an admin can remedy this via `ownerMint` to re-issue the missing supply. The more fundamental problem is that the surplus never returns to the vault: `totalAssets()` and share price are unchanged after the burn, so remaining stakers absorb the full loss via `realizeLosses` and receive nothing from the surplus. They are penalized twice: once when the vault's `HilBTC` balance is reduced by `realizeLosses`, and again when the surplus that should replenish the pool is instead destroyed.

Source: `issuance/src/vault/StakingVault.sol:351-388`.

Impact: The surplus burn permanently removes `HilBTC` supply without redeeming the base asset that backed it, leaving orphaned base asset in the Minter, but an admin can cover this via `ownerMint` to re-issue the missing supply and restore the 1:1 backing. The more immediate and non-recoverable impact is unfair value distribution: remaining stakers absorb the full loss via `realizeLosses` and receive nothing from the surplus that should have replenished the pool. Users in cooldown take full downside from post-redeem losses with no upside from yield vested during the same period. The longer `cooldownDuration`, the wider the window for this asymmetry (up to 90 days). From PoC run: a user with `_initialAssets = 1038` receives `previewedAssets = 988`, surplus of 50 `HilBTC` units is burned, and remaining stakers see zero share-price gain; under correct behavior remaining stakers' value would rise by 41 units — that redistribution error is the core impact every time the path triggers.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

import "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {StakingVault} from "../../src/vault/StakingVault.sol";
import {IStakingVault} from "../../src/vault/StakingVault.sol";
import {Minter} from "../../src/minter/Minter.sol";
import {MockFeed} from "../../src/vault/MockFeed.sol";
import {MockERC20} from "../../mocks/MockERC20.sol";
import {MockPolicyEngine} from "../../src/kyc/MockPolicyEngine.sol";
import {Distributor} from "../../src/vault/Distributor.sol";
import {
    ERC1967Proxy
} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

contract H03_ClaimWithdrawBurn is Test {
```

```

StakingVault public vault;
Minter public minter;
MockFeed public mf;
MockERC20 public asset;
Distributor public distributor;

address public owner = address(this);
address public custodian = address(2);

address public userA = makeAddr("userA");
address public userB = makeAddr("userB");

uint256 internal constant INIT_STAKE = 2_000;

function setUp() public {
    vm.warp(1770880586);
    vm.startPrank(owner);

    asset = new MockERC20();
    MockERC20 baseAsset = new MockERC20();
    StakingVault implVault = new StakingVault();
    bytes memory initDataVault = abi.encodeWithSelector(
        StakingVault.initialize.selector,
        IERC20(address(asset)),
        "Staked TEST",
        "sTEST",
        owner,
        1_000,
        address(0)
    );
    ERC1967Proxy proxyVault = new ERC1967Proxy(
        address(implVault),
        initDataVault
    );
    vault = StakingVault(address(proxyVault));
    mf = new MockFeed();

    Minter impl = new Minter();
    MockPolicyEngine policyEngine = new MockPolicyEngine();

    bytes memory initData = abi.encodeWithSelector(
        Minter.initialize.selector,
        owner,
        address(baseAsset),
        address(asset),
        custodian,
        owner,
        owner,
        address(0),
        address(policyEngine),
        vault
    );
    ERC1967Proxy proxy = new ERC1967Proxy(address(impl), initData);
    minter = Minter(address(proxy));

    Distributor distrImpl = new Distributor();
    bytes memory initDataDistr = abi.encodeWithSelector(
        Distributor.initialize.selector,
        owner,
        mf,
        minter,
        8 hours
    );
};

```

```

ERC1967Proxy proxyDistr = new ERC1967Proxy(
    address(distrImpl),
    initDataDistr
);
distributor = Distributor(address(proxyDistr));

vault.setDistributor(address(minter));
minter.requestDistributorChange(address(distributor));
vm.warp(block.timestamp + 2 days);
minter.setDistributor();

vault.setCooldownDuration(0);
asset.mint(owner, 2_000);
asset.approve(address(vault), 2_000);
vault.stake(1_000, "");

vault.setCooldownDuration(7 days);
vm.stopPrank();

asset.mint(userA, 1_000);
asset.mint(userB, 10_000);
}

function _stake(address who, uint256 amount) internal {
    vm.startPrank(who);
    asset.approve(address(vault), amount);
    vault.stake(amount, "");
    vm.stopPrank();
}

function test_PoC_claimWithdrawBurnsSurplusOnPostRedeemLoss() public {
    _stake(userA, 1_000);
    _stake(userB, 10_000);

    asset.mint(address(vault), 500);

    uint256 aShares = vault.balanceOf(userA);
    uint256 previewAtRedeem = vault.previewRedeem(aShares);
    assertGt(previewAtRedeem, 1_000);

    vm.prank(userA);
    vault.redeem(aShares);

    IStakingVault.UserCooldown memory cd = vault.getCooldown(userA);
    uint256 initialAssets = cd.underlyingAmount;
    uint256 initialShares = cd.underlyingShares;
    assertEq(initialAssets, previewAtRedeem);

    address holder = vault.tokensHolder();
    assertEq(asset.balanceOf(holder), initialAssets);

    // Simulate the realizeLosses-dead fix being applied so realizeLosses works.
    vm.prank(address(vault));
    asset.approve(address(minter), type(uint256).max);

    vm.prank(owner);
    distributor.realizeLosses(600);

    uint256 previewAfterLoss = vault.previewRedeem(initialShares);
    assertLt(previewAfterLoss, initialAssets);

    uint256 holderBeforeClaim = asset.balanceOf(holder);
    uint256 vaultBeforeClaim = asset.balanceOf(address(vault));

```

```

uint256 supplyBeforeClaim = asset.totalSupply();
uint256 totalAssetsBeforeClaim = vault.totalAssets();
uint256 userBSharesBefore = vault.balanceOf(userB);
uint256 userBValueBefore = vault.previewRedeem(userBSharesBefore);

skip(7 days + 1);

vm.prank(userA);
vault.claimWithdraw(userA);

uint256 userABal = asset.balanceOf(userA);
assertEq(userABal, previewAfterLoss);
assertLt(userABal, initialAssets);

uint256 surplus = initialAssets - previewAfterLoss;
assertGt(surplus, 0);
assertEq(asset.totalSupply(), supplyBeforeClaim - surplus,
    "surplus was burned from global supply");

assertEq(asset.balanceOf(holder), holderBeforeClaim - initialAssets);
assertEq(asset.balanceOf(address(vault)), vaultBeforeClaim);
assertEq(vault.totalAssets(), totalAssetsBeforeClaim,
    "totalAssets unchanged: surplus did NOT benefit remaining stakers");

uint256 userBValueAfter = vault.previewRedeem(userBSharesBefore);
assertEq(userBValueAfter, userBValueBefore,
    "userB did not benefit from the burned surplus");

emit log_named_uint("userA escrowed (_initialAssets)", initialAssets);
emit log_named_uint("userA received (previewedAssets)", userABal);
emit log_named_uint("surplus BURNED", surplus);
}
}

```

Expected output: test_PoC_claimWithdrawBurnsSurplusOnPostRedeemLoss passes. Logs show _initialAssets = 1038, previewedAssets = 988, surplus 50 burned, userB value before and after claim unchanged at 9885. A companion test demonstrates that under correct behavior (surplus retained), userB value would rise to 9926 - a 41-unit gain that the burn path destroys.

Recommended Mitigation: Two approaches, in order of preference:

Option A - Share escrow: Replace asset escrow with share escrow.

- At redeem(): **transfer** Alice's shares to TokensHolder instead of burning them. Do not move any tokens. Record cooldowns[msg.sender].underlyingShares as before.
- At claimWithdraw(): call _burn(address(tokensHolder), underlyingShares) directly (internal _burn requires no allowance), then pay previewRedeem(underlyingShares) from the vault's own balance to the receiver.

Because the shares sit in TokensHolder throughout the cooldown they remain in totalSupply, so realizeLosses automatically distributes any loss proportionally across all participants — exactly as if the user had redeemed after the loss. Example: vault 1000 tokens / 1000 shares, Alice transfers 100 shares to TokensHolder, realizeLosses(100) fires → price = 0.9; Alice receives 90, remaining stakers hold 810. No surplus exists, nothing is burned incorrectly, and neither party is double-penalised. This also eliminates the asymmetric floor: Alice participates symmetrically in yield upside and loss downside during cooldown. Alice's shares leave her balance immediately so maxRedeem(alice) naturally returns 0 with no extra guard needed. TokensHolder no longer needs to hold HiBTC for redemptions — the vault pays directly from its balance at claim time.

Option B - Lock exchange rate at redeem time: Keep asset escrow but fix the amount at redeem() and never recompute it.

- At redeem(): burn shares immediately, escrow _initialAssets = previewRedeem(shares) to TokensH-

older as today, but do **not** call `previewRedeem` again at claim time.

- At `claimWithdraw()`: pay the user exactly `_initialAssets` from `TokensHolder`. No surplus branch, no burn.

This eliminates the surplus burn and the orphaned collateral entirely. The tradeoff is that the exiting user is fully insulated from post-redeem losses, losses are not socialized across cooldown holders, so remaining stakers absorb slightly more than under Option A. It is simpler to implement and reason about, and removes the asymmetric floor by giving the user a fixed, predictable payout.

Syntetika: Fixed in commit [8ed5c9d](#)

Cyfrin: The normal cooldown exit correctly passes `userCooldown.underlyingShares` directly to `_withdraw`, burning all escrowed shares regardless of price movement:

```
_withdraw(address(tokensHolder), receiver, address(tokensHolder), assets,  
↪ userCooldown.underlyingShares);
```

The **early exit** path does not:

```
uint256 newShares = previewWithdraw(netWithdraw);  
uint256 earlyExitSharesFee = previewWithdraw(earlyExitFee);  
_withdraw(tokensHolder, receiver, tokensHolder, netWithdraw, newShares);  
_withdraw(tokensHolder, feeRecipient, tokensHolder, earlyExitFee, earlyExitSharesFee);
```

Total shares burned = `previewWithdraw(assets)` where `assets = min(_initialAssets, previewedAssets)`.

When yield accrued during cooldown (`previewedAssets > _initialAssets`), `assets = _initialAssets`. At the higher current price `P`:

```
previewWithdraw(_initialAssets at P) < previewWithdraw(_initialAssets at P) = underlyingShares
```

The shortfall, `underlyingShares - newShares - earlyExitSharesFee`, remains in `TokensHolder` as permanently stranded shares. `delete $.cooldowns[msg.sender]` wipes the record so they can never be claimed.

Concrete example: vault has 1100 tokens / 1000 shares (Alice's 500 in `TokensHolder`, Bob's 500), price = 1.1, `_initialAssets = 500` (locked in when price was 1.0). Early exit with 5% fee: burned 453 shares, 47 left in `TokensHolder` forever. After the claim, vault has 600 tokens / 547 shares. Bob's 500 shares are worth ~548 instead of 600. Those 47 zombie shares permanently dilute remaining stakers.

The loss scales with: the amount of yield earned × the fee percentage × how early the user exits. With 90-day cooldowns and early exit enabled, this is a realistic and repeating scenario.

Fix: in the early exit branch, split `userCooldown.underlyingShares` proportionally by the fee ratio rather than recomputing from `previewWithdraw`:

```
uint256 earlyExitSharesFee = userCooldown.underlyingShares * earlyExitFee / assets;  
uint256 newShares = userCooldown.underlyingShares - earlyExitSharesFee;
```

This burns all escrowed shares regardless of how the price moved, matching the behaviour of the normal exit path.

Syntetika: Fixed in commit [eeb4e91](#)

Cyfrin: Verified.

7.2.2 `HilToken::_update` inverted waiting-period check + permissionless `HilToken::requestTransfer` enables single-tx force-burn of any holder's balance

Description: Two coupled defects in the force-transfer / force-burn mechanism:

(a) `requestTransfer(from, to, amount)` is permissionless - any caller overwrites `$.pendingTransfer` with arbitrary values. Every other `request*` function in scope is admin-gated.

(b) The `_update` owner-bypass waiting-period check uses `>` instead of `<=`:

```

require(
  $.pendingTransfer.to == to &&
  $.pendingTransfer.from == from &&
  $.pendingTransfer.amount == amount &&
  $.pendingTransfer.timestamp + $.requiredWaitingPeriod >
  block.timestamp,
  TransfersNotAllowed()
);

```

Same inversion pattern as on the `ownerMint` check; the check passes only while the window is still open - no actual delay.

Impact: Instant confiscation / destruction of any holder's tokens by the owner key. Griefing variant: because `requestTransfer` is permissionless, any user can overwrite the pending slot, bricking any legitimate owner force-transfer flow. Downstream: burned user cannot redeem baseAsset via Minter; permanent fund loss for affected users. For CCT future-work, the same path destroys bridge pool inventory.

Recommended Mitigation:

- Gate `requestTransfer` with `onlyRole(DEFAULT_ADMIN_ROLE)` or `onlyOwner`
- Flip the operator in `_update` to `<=` and add `$.pendingTransfer.timestamp != 0` guard

Syntetika: Fixed in commit [0c8b861](#)

Cyfrin: Verified. Operator flipped and `requestTransfer` gated to `DEFAULT_ADMIN_ROLE`.

7.2.3 `Distributor::distributeYield` **permissionless + no yieldAmount validation - griefing burns rate-limit slot**

Description: `distributeYield` is permissionless (by design, rate-limited by `timeLock`). It forwards `feed.getYieldAmount()` unchecked. If the feed transiently returns 0 (e.g., between `updateFeed` and feed heartbeat, or by design in low-yield periods), an attacker or any bot can:

1. Call `distributeYield()` - `yieldAmount = 0` minted, `lastDistribution = block.timestamp`.
2. Legitimate distribution is now locked out for `timeLock` seconds. The real yield accrued during that window is lost (feed is non-cumulative).

Additionally: on the first call, `lastDistribution == 0` makes the rate-limit trivially pass, so a griever can race the first legitimate call.

Source: `issuance/src/vault/Distributor.sol:122-131`.

Impact: Griefing that deprives stakers of one `timeLock` epoch of yield per occurrence. Repeatable on every feed transition. Feed integration is future-work (known issue), but the permissionless-caller design with no `yieldAmount > 0` check is a contract-side bug that survives any feed choice.

Recommended Mitigation: Add `require(yieldAmount > 0, ZeroYield())`. Consider gating `distributeYield` behind a loose role (keeper). Initialize `lastDistribution = block.timestamp` in `initialize` so first-call also respects the cadence.

Syntetika: Fixed in commit [ace7bea](#)

Cyfrin: Verified. Call reverts if yield is 0.

7.3 Low Risk

7.3.1 StakingVault::withdraw, redeem **silently ignore ERC-4626 receiver and owner parameters - causes fund loss for aggregators**

Description: Both functions override the standard ERC-4626 3-argument signatures but declare `receiver` and `owner` as unnamed parameters and silently use `msg.sender` internally for (1) share burn (via `_redeemTo` / `_withdrawTo` which call `_withdraw(msg.sender, ..., msg.sender, ...)`), (2) cooldown key (`$.cooldowns[msg.sender]`), and (3) later claim authority (`claimWithdraw(receiver)` called by the original redeemer).

ERC-4626 spec requires burning shares from `owner` (spending `msg.sender`'s allowance) and sending assets to `receiver`. Any ERC-4626-compliant aggregator/router (Yearn, ERC-4626 alliance routers, Euler Earn, etc.) that calls `vault.redeem(shares, user, user)` on behalf of a user will either revert with `ERC4626ExceededMaxRedeem` if the aggregator holds no vault shares itself, or silently burn the aggregator's OWN pooled shares keyed to the aggregator rather than to the user - losing funds of other aggregator users and stranding this user's funds in the aggregator's cooldown slot.

The deposit/mint path correctly honors `receiver`, making the asymmetry particularly dangerous: value enters via the standard interface but cannot leave through it.

Source: `issuance/src/vault/StakingVault.sol:297-344`.

Impact: Any ERC-4626 integration loses funds or DoSes. Breaks compatibility with all standard router/aggregator integrations. Future CCT integration (share lock&release) also becomes uncertain since destination chain wrappers typically rely on canonical 4626 semantics.

Recommended Mitigation: Either honor the spec - use `receiver` and `owner` with proper `_spendAllowance(owner, msg.sender, shares)` and key cooldowns by `owner` - or remove the 3-arg overrides entirely so the inherited OZ implementations either revert loudly or the contract doesn't advertise 4626 compliance.

Syntetika: Fixed in commit [44fa99e](#)

Cyfrin: `claimWithdraw` permanently reverts after cooldown elapses when `earlyExitEnabled = true`

When `earlyExitEnabled = true` and a user's cooldown has fully elapsed, `claimWithdraw` falls into the `else if ($.earlyExitEnabled)` branch (not the `!earlyExitEnabled && elapsed` branch). `getEarlyExitAmount` returns (`fee=0`, `netWithdraw=assets`). The second `_withdraw(... earlyExitFee=0, earlyExitSharesFee=0)` then hits `if (assets < $.minAssetsAmount)` revert `AmountBelowLimit()` at line 668. With `minAssetsAmount = 1000` (deployment default), every elapsed cooldown is unclaimable while `earlyExitEnabled` is on. Users must either pay the early-exit fee before cooldown ends, or wait for admin to toggle the flag off.

Fix: add `block.timestamp >= userCooldown.cooldownEnd` check before the early-exit branch, OR skip the second `_withdraw` when `earlyExitFee == 0`.

Syntetika: Fixed in commit [6fdc89d](#)

Cyfrin: In the cooldown path: `_transfer` is OZ's internal ERC-20 function, it moves balances directly with no allowance check. If `msg.sender != owner`, the shares are transferred from `owner` to `TokensHolder` without `owner`'s consent. Anyone can call `vault.redeem(bob_shares, anyAddress, bob)` to force Bob's shares into cooldown and reset his cooldown timer.

The fix should add an explicit allowance check at the top of the cooldown branch:

```
if (msg.sender != owner) {
    _spendAllowance(owner, msg.sender, shares);
}
```

before the `_transfer` call, matching what OZ's `_withdraw` does in the no-cooldown path.

Syntetika: Fixed in commit [f221196](#)

Cyfrin: When `earlyExitEnabled = true` and a user exits early late in their cooldown window, `earlyExitFee` can be non-zero but smaller than `minAssetsAmount`. The new guard `if (earlyExitFee >= $.minAssetsAmount)`

skips the second `_withdraw`, but `earlyExitSharesFee` shares are already excluded from the first burn, they sit in `TokensHolder` permanently with no cooldown record pointing to them.

Fix (Option A): Burn `earlyExitSharesFee` unconditionally; only gate the asset transfer on `earlyExitFee >= $.minAssetsAmount`. Fix (Option B): When the fee would be below the floor, treat it as zero, set `netWithdraw = assets` and burn all underlyingShares.

Syntetika: Fixed in commit [34eb001](#)

7.3.2 `Minter::ownerMint` inverted waiting-period check enables single-tx unbacked mint; chained with self-whitelist drains all `Minter` `baseAsset`

Description: The 2-step admin pattern is intended to force `requiredWaitingPeriod` (≥ 1 day) between `requestOwnerMint` and `ownerMint`. All other 2-step sites in `Minter` use `timestamp + requiredWaitingPeriod <= block.timestamp`; `ownerMint` alone uses $>$:

```
require(
    $.ownerMintRequest.timestamp + $.requiredWaitingPeriod >
        block.timestamp,
    WaitingPeriodNotElapsed()
);
```

The operator is inverted: the `require` passes while the waiting period has NOT yet elapsed and reverts afterwards. A malicious admin (or leaked admin key) can call `requestOwnerMint(amount, attacker)` and `ownerMint()` in the same block. Chained with `Minter::whitelistAddress` (single-step admin) and `Minter::redeem` (whitelist-gated on `msg.sender` only), an attacker with `DEFAULT_ADMIN_ROLE` drains in one block: (1) `requestOwnerMint(huge, attacker) + ownerMint()`; (2) `whitelistAddress(attacker, true)`; (3) `Minter::redeem(amount)` - `spender==from` skips allowance; `baseAsset.safeTransfer(attacker, amount)` drains all `Minter` `baseAsset` reserves.

Source: `issuance/src/minter/Minter.sol:232-246`.

Impact: Single-tx drain of all on-chain `baseAsset` held in `Minter` plus permanent inflation of `HilSyntheticToken` supply under a compromised admin key. Directly violates client's stated 2-step invariant. Off-chain custodian-held `baseAsset` is unaffected, but the 1:1 peg is destroyed instantly. Also breaks the honest flow: after the intended 1-day wait, the same call reverts permanently until a fresh request.

Recommended Mitigation:

```
require(
    $.ownerMintRequest.timestamp != 0 &&
    $.ownerMintRequest.timestamp + $.requiredWaitingPeriod <= block.timestamp,
    WaitingPeriodNotElapsed()
);
```

Also validate `to != address(0)` and `amount > 0` in `requestOwnerMint`. Note: the per-site fix patches the 2-step flow itself, but the architectural `grantRole` bypass separately makes the 2-step surface bypassable via direct `grantRole`.

Syntetika: Fixed in commit [dc0cfc7](#)

Cyfrin: Verified.

7.3.3 Direct `grantRole`, `revokeRole` bypasses the 2-step role mechanism across `Minter`, `HilToken`, `StakingVault`, `Distributor`

Description: The protocol's stated threat model - "the 2-step admin system prevents a leaked admin private key from destroying the protocol in a single transaction" - relies on role transitions going through per-contract `request/execute` pairs (e.g. `HilToken::requestMinterChange` -> `wait` -> `setMinter`). Each setter pair updates both a storage pointer and the corresponding OZ role behind a 1-day waiting period.

However, the role itself is grantable through the standard OpenZeppelin surface: `AccessControlUpgradeable::grantRole` is public, gated only by `onlyRole(getRoleAdmin(role))`, and every custom role's admin is `DEFAULT_ADMIN_ROLE`. No in-scope contract overrides `grantRole/revokeRole`, calls `_setRoleAdmin`, or uses `AccessControlDefaultAdminRulesUpgradeable`. Because role membership - not the storage pointer - is what authorizes `HilToken::mint`, `Minter::transferToCustody`, `pause`, `Distributor::realizeLosses`.

`DEFAULT_ADMIN_ROLE` itself is the least protected role - no delay, no 2-step, no acceptance step. A compromised admin can grant it to an attacker in one tx, or renounce it and permanently brick every `onlyRole(DEFAULT_ADMIN_ROLE)` function (including `_authorizeUpgrade`, freezing the UUPS proxy forever).

Proof - `HilToken` mint-and-drain, single block, single leaked admin key:

<code>Tx1: HilToken.grantRole(MINTER_ROLE, attackerEOA)</code>	<code>// passes: onlyRole(DEFAULT_ADMIN_ROLE)</code>
<code>Tx2: HilBTC.mint(attackerEOA, type(uint128).max)</code>	<code>// passes: onlyRole(MINTER_ROLE)</code>
<code>Tx3: Minter.whitelistAddress(attackerEOA, true)</code>	<code>// single-step admin</code>
<code>Tx4: Minter.redeem(type(uint128).max)</code>	<code>// drains wBTC/USDC in Minter</code>

The same pattern unlocks every other role-gated capability across `Minter`, `StakingVault`, `Distributor`.

Off-chain monitoring built around the 2-step events (`MinterChangeRequested/MinterChanged`) misses this entirely - only the raw OZ `RoleGranted` event fires, and `$.minter/$.distributor` pointers stay stale.

Sources: `Minter.sol:15,34-41`; `HilToken.sol:7,126-141`; `StakingVault.sol:26,138-148`; `Distributor.sol:4`; `deposit-registry/contracts/ComplianceCheckerUpgradeable.sol:6`.

Impact: Categorical violation of the stated "leaked admin key cannot destroy the protocol in a single tx" invariant. Every role-gated capability is reachable in a single transaction after an admin-key compromise.

Recommended Mitigation: Gate the role grant itself, not just the storage pointer. Any of the following (ideally combined):

1. Override `grantRole/revokeRole` for protected roles to either reject direct calls (forcing management through the 2-step `set*` functions) or enforce the same waiting-period check before calling `super`.
2. Migrate `DEFAULT_ADMIN_ROLE` handling to `AccessControlDefaultAdminRulesUpgradeable` - 2-step + delay on admin transfer, blocks naive `renounceRole`.
3. Use `_setRoleAdmin` to route every custom role through a `ROLE_ADMIN_ROLE` held by a timelock contract.
4. Hold the admin key in a multisig + on-chain timelock at deployment level, so the timelock provides the delay the 2-step pattern was intended to provide.

Syntetika: Fixed in commit [5a7840a](#)

Cyfrin: Verified. 2-step change process for all roles implemented.

7.3.4 `Minter::totalDeposits` accounting drifts from both reserves and synthetic supply across every privileged path

Description: `totalDeposits` is documented as "total amount of base asset currently held in the Minter." Mutation table:

Path	totalDeposits	baseAsset change	hilSynth change
<code>mint</code>	+amount	+amount	+amount
<code>redeem</code>	-amount	-amount	-amount
<code>ownerMint</code>	+amount	0	+amount
<code>distributeYield</code>	+amount	0	+amount
<code>transferToCustody</code>	0	-amount	0
<code>realizeLosses</code>	0	0	-amount

Consequences: `totalDeposits != baseAsset_held + custody` (broken by `ownerMint`, `distributeYield`, `realizeLosses`) and `totalDeposits != hilSynth.totalSupply()` (broken by `realizeLosses`). Additionally, `transferToCustody` has no caps - operator can drain all Minter reserves in one call, while `totalDeposits` remains stale. Users redeeming afterwards get `ERC20InsufficientBalance` on the `baseAsset` transfer, `DoSing` redemption despite the accounting showing full backing.

Source: `issuance/src/minter/Minter.sol:244, 259, 391-398, 431-440`.

Impact: Off-chain dashboards, PoR attestations, and any future contract logic that reads `totalDeposits` (e.g., CCT hooks, cap enforcement) receive misleading values. Redemption is `DoSable` by operator action with no on-chain safeguard. Related to the unbacked-mint issue - both stem from unreconciled accounting.

Recommended Mitigation: Decide the semantic - live reserves vs. outstanding liability - and enforce consistently. Preferred: remove `totalDeposits` entirely and derive from `baseAsset.balanceOf(address(this))`. Alternatively, add a minimum reserve floor enforced on `transferToCustody`: `require(baseAsset.balanceOf(this) - amount >= totalDeposits * minReserveBps / BPS)`.

Syntetika: Fixed in commit [bae5a23](#)

Cyfrin: Verified. `totalDeposits` removed.

7.3.5 Minter::mint docstring vs code: recipient is never checked against whitelist

Description: The NatSpec for `Minter::mint` states:

`@dev Requires both sender and recipient to be whitelisted.`

The implementation applies only `onlyWhitelisted(msg.sender)`:

```
function mint(
    address to,
    uint256 amount
) external onlyWhitelisted(msg.sender) nonReentrant whenNotPaused {
    MinterStorage storage $ = _getMinterStorage();
    $.baseAsset.safeTransferFrom(msg.sender, address(this), amount);
    $.hilSyntheticToken.mint(to, amount); // ← to is never checked
    $.totalDeposits += amount;
    emit Minted(to, amount);
}
```

`onlyWhitelisted` either fast-paths for a manually whitelisted address or runs `PolicyEngine.run()` for non-whitelisted callers. The modifier is called once, with `msg.sender`. The `to` parameter is never passed to `onlyWhitelisted` or any equivalent compliance gate. `HilToken::_update` subsequently checks only the `_blacklisted` mapping (not the whitelist) for `from`, `to`, and `msg.sender` — so minting to a non-blacklisted, non-whitelisted address succeeds without triggering any compliance check.

A whitelisted caller can therefore:

1. Call `Minter::mint(nonCompliantAddress, amount)` — depositing `baseAsset` and receiving amount `HilBTC/HilUSD` credited to `nonCompliantAddress`.
2. `nonCompliantAddress` now holds regulated synthetic assets despite never being screened by the whitelist or `PolicyEngine`.

The `onlyWhitelisted` check on transfers via `HilToken::requestTransfer / _update` is the only downstream gate, but it applies to *transfers*, not to the initial mint — so tokens land in a non-compliant wallet before any transfer-time checks can fire.

Impact:

- Synthetic `HilBTC/HilUSD` can be issued to wallets that have not undergone any KYC/AML screening, directly contradicting the protocol's stated compliance model.
- A whitelisted user (e.g., one institution's trader) can mint or transfer to an arbitrary counterparty without that counterparty satisfying any onboarding requirement.

- Depending on regulatory context, issuance of regulated synthetic assets to unscreened recipients may expose the protocol operator to compliance liability independent of technical severity.

Mitigation:

Add `onlyWhitelisted(to)` to `Minter::mint` alongside the existing `onlyWhitelisted(msg.sender)` check:

```
function mint(
    address to,
    uint256 amount
) external onlyWhitelisted(msg.sender) onlyWhitelisted(to) nonReentrant whenNotPaused {
```

Alternatively, add an explicit `require(isAddressWhitelisted(to) || policyCheck(to), RecipientNotCompliant())` if the `onlyWhitelisted` double-modifier approach conflicts with gas or `PolicyEngine` assumptions.

Additionally, implement whitelist checks in `HilToken::_update` to disallow transfers to non-whitelisted users.

Syntetika: Fixed in commit [38c2f5c](#): Whitelist required only for minting - transferring should without wl restriction.

Cyfrin: Verified. Comment changed to remove mentioning whitelisting.

7.3.6 Events / event-argument inconsistencies across contracts

Description: Multiple event-related inconsistencies were found across the in-scope contracts. Each on its own is low-severity, but collectively they weaken off-chain monitoring, indexer reliability, and post-mortem forensics. Grouped as one finding because the root cause and mitigation class is uniform (add / fix events); individually listed below so each can be resolved concretely:

Missing completion events on 2-step admin transitions

1. `Minter::setOperator` (`issuance/src/minter/Minter.sol:278-286`), `Minter::setDistributor` (`301-319`), `Minter::setCustodian` (`334-347`), `Minter::setPauser` (`362-375`) — only the request-side event fires (`DistributorChangeRequested` etc.); completion is silent. `setOperator` is one-step and emits no event at all.
2. `HilToken::setMinter` (`issuance/src/token/HilToken.sol:126-141`) — only `MinterChangeRequested` fires at request time; completion of the `MINTER_ROLE` transition is silent.
3. `HilToken::_update` owner-bypass branch (`issuance/src/token/HilToken.sol:218-235`) — consumes the `pendingTransfer` struct silently; only the standard ERC-20 `Transfer` fires, with no semantic that this was a forced-transfer execution of a prior `TransferRequested`.

Missing events on state-changing admin / governance parameters

4. `Minter::updateWaitingPeriod` (`issuance/src/minter/Minter.sol:204-213`) — mutates the waiting period that gates all five 2-step admin transitions, with no event.
5. `HilToken::updateWaitingPeriod` (`issuance/src/token/HilToken.sol:180-189`) — same class as (4), on `HilToken`.
6. `Distributor::rescueERC20` (`issuance/src/vault/Distributor.sol:142-151`) — admin pulls arbitrary ERC-20 balance to an arbitrary recipient; no contract-level event (only the ERC-20 `Transfer`).
7. `Minter::_authorizeUpgrade` — no event emitted on successful upgrade; upgrade execution is invisible to off-chain monitors that track the event log.

Missing events on state-changing yield / loss paths

8. `Distributor::distributeYield` (`issuance/src/vault/Distributor.sol:122-131`) — mutates `$.lastDistribution`, pulls `yieldAmount` from the feed; emits nothing at the `Distributor` layer.
9. `Distributor::realizeLosses` (`issuance/src/vault/Distributor.sol:135-140`) — forwards to `Minter::realizeLosses` with no `Distributor`-level event.

10. `TokensHolder::withdraw` (issuance/src/helpers/TokensHolder.sol:34-36) — transfers HILBTC out of custody with no event.
11. `StakingVault::_withdraw` last-user branch (issuance/src/vault/StakingVault.sol:597-605) — when the vault empties to `DEAD_SHARES`, unvested yield is zeroed and transferred to `owner()`; no event marks the claw-back.

Silent branches in existing emit paths

12. `StakingVault::redeem`/`StakingVault::withdraw` cooldown-initiation branches (issuance/src/vault/StakingVault.sol:317, 325-344) emit only the generic `Unstaked(msg.sender, assets)` which also fires on the `cooldownDuration == 0` instant-exit path. Off-chain consumers cannot distinguish "pending cooldown" from "instant exit" from the event alone.

Inaccurate or incomplete event arguments

13. `StakingVault::claimWithdraw` emits `WithdrawClaimed(msg.sender, assets)` (issuance/src/vault/StakingVault.sol:317) where `assets = min(previewedAssets, _initialAssets)` — the pre-fee total. When the early-exit branch is taken, the user actually receives `withdrawAmount < assets` and the fee is diverted to `$.earlyExitFeeRecipient`; the event's `assets` field overstates net receipt. The event also omits the receiver (distinct from `msg.sender`) and the burned residual (`_initialAssets - assets`).
14. `Minted(address(_stakingVault), amount)` in `Minter::distributeYield` (issuance/src/minter/Minter.sol:262) — misleading because the ERC-20 Transfer that precedes it shows `from=0, to=Minter`, then the vault-as-recipient framing of the `Minted` event creates dual off-chain interpretations of a single mint-and-forward flow.
15. `OwnerMintRequested(uint256 amount)` declared at `issuance/src/interfaces/minter/IMinter.sol:31` — no `to` field and no indexed. The `MintRequest` struct stores `amount` and `to`; consumers can't filter by recipient without an RPC state read.
16. `LossesRealized(lossesAmount)` (`Minter.sol:439`) and `FundsTransferredToCustody(amount, custodian)` (`Minter.sol:397`) — neither event includes the caller address. Role-gated functions with multiple possible role-holders need caller attribution for forensics.
17. `HilToken::requestTransfer` emits `TransferRequested(to, amount)` (issuance/src/token/HilToken.sol:203) — omits `from`, so off-chain indexers cannot tell whose tokens a pending transfer concerns. Additionally the function silently overwrites any previously pending transfer; callers get no signal of the invalidated request.
18. `BlacklistableUpgradeable::updateBlacklist` emits `BlacklistChanged(newBlacklist)` (issuance/src/helpers/BlacklistableUpgradeable.sol:96-102) — omits the previous blacklist, making blacklist rotation unauditably from event logs alone.

Missing indexed on address parameters

19. Six `StakingVault` admin-setter events omit indexed on address fields (notably `EarlyExitFeeRecipientUpdated(address newRecipient)` at `IStakingVault.sol:69`). `AddressWhitelisted(user, allowed)` in `Whitelist.sol` similarly omits indexed on `user`. Both are inconsistent with `Distributor's FeedUpdated(IFeed indexed _feed)` style.

Recommended Mitigation: Per-item, concrete fixes:

1. Add and emit event `OperatorChanged(address indexed previousOperator, address indexed newOperator), event DistributorChanged(...), event CustodianChanged(...), event PauserChanged(...)`.
2. Add and emit event `MinterChanged(address indexed previousMinter, address indexed newMinter)` in `HilToken::setMinter`.
3. Add and emit event `TransferExecuted(address indexed from, address indexed to, uint256 amount)` inside the owner-bypass branch before clearing the `pendingTransfer` struct. 4–5. Add and emit event `WaitingPeriodUpdated(uint256 previousValue, uint256 newValue)` in both `updateWaitingPeriod` implementations.
4. Add and emit event `ERC20Rescued(address indexed token, address indexed to, uint256 amount)` in `rescueERC20`.

5. Add and emit event `Upgraded(address indexed implementation) in _authorizeUpgrade`.
6. Add and emit event `YieldPulled(uint256 yieldAmount, uint256 timestamp) in Distributor::distributeYield` after `$.lastDistribution = block.timestamp`.
7. Add and emit event `LossesForwarded(uint256 amount) in Distributor::realizeLosses`.
8. Add and emit event `Withdrawn(address indexed to, uint256 amount) in TokensHolder::withdraw`.
9. Add and emit event `UnvestedYieldReclaimed(address indexed owner, uint256 amount) inside the remainingUnvested > 0 branch before the safeTransfer`.
10. Add and emit event `CooldownInitiated(address indexed user, uint256 assets, uint256 shares, uint104 cooldownEnd) inside the cooldownDuration != 0 branch of both redeem and withdraw`.
11. Widen `WithdrawClaimed` to event `WithdrawClaimed(address indexed user, address indexed receiver, uint256 totalAssets, uint256 receivedAmount, uint256 fee) OR` emit a distinct event `EarlyExitFeeCharged(address indexed user, address indexed feeRecipient, uint256 fee)` in the early-exit branch.
12. Change `Minted(to, amount) in Minter::distributeYield` to either `YieldMinted(address indexed vault, uint256 amount)` or add a new event distinct from user-initiated `Minted` so that downstream indexers can filter yield vs user-mint flows.
13. Change declaration to event `OwnerMintRequested(address indexed to, uint256 amount)` and update the emit at `Minter.sol:227` to pass both arguments.
14. Widen both events to include the caller: event `LossesRealized(address indexed caller, uint256 losses)`, event `FundsTransferredToCustody(address indexed caller, uint256 amount, address indexed custodian)`.
15. Change `TransferRequested` to include from: event `TransferRequested(address indexed from, address indexed to, uint256 amount)`.
16. Change `BlacklistChanged` to include the old address: event `BlacklistChanged(address indexed oldBlacklist, address indexed newBlacklist)`.
17. Mark `EarlyExitFeeRecipientUpdated.newRecipient` and `AddressWhitelisted.user` as indexed; review remaining `StakingVault` admin-setter events for parity with the `Distributor` convention.

Syntetika: Fixed in commits: [1a68840](#), [2ace59d](#), [857171a](#), [b980786](#), [8c8f4de](#), and [00fbf69](#)

Cyfrin: Verified.

7.3.7 `HilToken` admin blacklisting `TokensHolder` or `StakingVault` in its blacklist freezes every vault exit - cross-contract blacklist DoS by a single `HilToken` admin key

Description: `HilToken` and `StakingVault` each maintain their own `BlacklistableUpgradeable` storage (namespaced per contract). The `HilToken` blacklist (initially `_initialAdmin` on `HilToken` deploy) can blacklist ANY address in `HilToken`'s mapping - including protocol-internal addresses like `StakingVault` or `TokensHolder`. Downstream effect:

- `StakingVault::_withdraw` transfers `HilToken` to `TokensHolder`: `HilToken::_update(vault, tokensHolder, ...)` runs `notBlacklisted(to=tokensHolder)` - if `tokensHolder` is blacklisted in `HilToken`, all redeem/withdraw revert.
- `claimWithdraw` calls `tokensHolder.withdraw(receiver, assets)` which calls `HilToken.transfer: _update(tokensHolder, receiver, ...)` runs `notBlacklisted(from=tokensHolder)` - reverts if blacklisted.
- `claimWithdraw` also calls `IHilToken(asset()).burn(...)` when `assets < _initialAssets`; that `_update(stakingVault, 0, ...)` runs `notBlacklisted(from=stakingVault)` - reverts if `StakingVault` is blacklisted in `HilToken`.
- `Minter::realizeLosses` calls `HilToken.burnFrom(stakingVault, ...)` - same path; DoS if `StakingVault` blacklisted.

A single compromised HilToken admin/blacklister key can freeze the entire vault exit surface.

Sources: issuance/src/vault/StakingVault.sol:297-344, 351-388, 571-605; issuance/src/helpers/TokensHolder.sol:34-36.

Recommended Mitigation:

- Exclude protocol-internal addresses (vault, tokensHolder, minter) from being blacklistable in HilToken - maintain a hard-coded exclusion list
- Use a single unified blacklist registry shared across HilToken and StakingVault
- Document the operational invariant explicitly: "HilToken admin MUST NOT blacklist the StakingVault or TokensHolder addresses"

Syntetika: Fixed in commit [cd54186](#). Protected addresses will be set during deployment

Cyfrin: Verified. A list of non-blacklistable addresses introduced.

7.3.8 Missing runtime input validation on admin setters and creator-role functions

Description: Several admin setters validate only lower bounds, or nothing:

- `Minter::updateWaitingPeriod`, `HilToken::updateWaitingPeriod`: only $\geq \text{MIN_REQUEST_WAITING_PERIOD}$. Setting to `type(uint256).max` overflows every 2-step check, bricking upgrades. Because the upgrade path uses the same `waitingPeriod`, recovery via upgrade is impossible.
- `Distributor::updateTimeLock`: only > 0 . Extreme values freeze `distributeYield` and make `getUnvestedAmount` never-vest; Minter's `require(getUnvestedAmount() == 0)` guard then rejects all future yield calls.
- `StakingVault::setMinAssetsAmount`: no bounds. `type(uint256).max` reverts every deposit and new unstake with `AmountBelowLimit`.

Impact: Under a compromised admin, DoS of upgrades, yield pipeline, deposits/withdrawals, and onboarding. Some variants are unrecoverable without upgrade because the setter that protects the cap is itself affected by the cap.

Recommended Mitigation: Add reasonable upper bounds, e.g. `MAX_REQUEST_WAITING_PERIOD = 30 days`, `MAX_TIME_LOCK = 30 days`, `MAX_MIN_ASSETS_AMOUNT = 10 ** asset.decimals()`, `MAX_CHALLENGE_PERIOD = 30 days`.

Syntetika: Fixed in commit [33bc4ea](#). For compliant deposit registry - contract is deprecated, so OOS

Cyfrin: Verified.

7.3.9 BlacklistableUpgradeable uses single-step OwnableUpgradeable — instant ownership transfer bypasses all owner()-gated invariants

Description: issuance/src/helpers/BlacklistableUpgradeable.sol:10, issuance/src/token/HilToken.sol:169,211

BlacklistableUpgradeable inherits OZ's OwnableUpgradeable (single-step), not Ownable2StepUpgradeable. The `owner()` role carries elevated privileges in every HilToken derivative:

- `burnFrom`: allowance check skipped when `msg.sender == owner()` (design intent — regulatory confiscation)
- `_update`: `owner()` bypasses the blacklist check for all transfers; the force-transfer (`pendingTransfer`) branch is only reachable when `msg.sender == owner()`

Because `transferOwnership(newOwner)` is instant, a compromised blacklister key can hand `owner()` to an attacker in a single transaction with no waiting period — the attacker immediately inherits both the `burnFrom` allowance-skip and the `_update` force-transfer capability. Separately, `renounceOwnership()` sets `owner()` to `address(0)`, permanently breaking the force-transfer mechanism and the `burnFrom` admin path with no recovery path short of an upgrade.

This is asymmetric with how the rest of the protocol protects sensitive roles: `MINTER_ROLE`, `DISTRIBUTOR_ROLE`, and `CUSTODIAN_ROLE` all use explicit 2-step request/execute patterns with 1-day waits, yet the `owner()` role — which can force-burn any holder's balance — has no such protection.

Impact: Under a compromised owner key, instant handoff to an attacker who can force-burn any holder's tokens or execute force-transfers without waiting period. `renounceOwnership()` permanently disables force-transfer and the `burnFrom` admin path. Requires malicious or negligent admin.

Recommended Mitigation: Replace `OwnableUpgradeable` with `Ownable2StepUpgradeable` so `transferOwnership` only nominates the new owner and requires `acceptOwnership()`. Override `renounceOwnership()` to revert, since removing the owner is irreversible and breaks core token mechanics. Consider extending the 2-step waiting period to match other role setters (1 day).

Syntetika: Fixed in commit [536b5d3](#)

Cyfrin: Verified.

7.3.10 `StakingVault::redeem, withdraw` missing zero-output guard in no-cooldown path

Description: Both `redeem` and `withdraw` contain an explicit zero-output guard — but only in the cooldown branch. When `cooldownDuration == 0` both functions return immediately from their internal helper, bypassing the check entirely.

`redeem` (lines 304–305 vs. 312–314):

```
if (_cooldownDuration == 0) {
    return _redeemTo(shares, msg.sender); // no zero-assets check on return value
}
// cooldown path:
assets = _redeemTo(shares, address($.tokensHolder));
if (assets == 0) { revert ZeroAmount(); } // guard present here only
```

`withdraw` (lines 329–330 vs. 337–339) is symmetric: `return _withdrawTo(assets, msg.sender)` with the `if (shares == 0) revert ZeroAmount()` guard only in the cooldown branch.

`_redeemTo` (line 547) checks `shares > 0` on the input but not the output. `previewRedeem` uses floor division — `shares.mulDiv(totalAssets + 1, totalSupply + 10**offset, Rounding.Floor)` — so for a small shares value against a high share price, `assets` rounds to zero. The internal `_withdraw` guard at line 589 (`if (assets < $.minAssetsAmount) revert AmountBelowLimit()`) is the only remaining backstop, but `minAssetsAmount` is configurable to zero via `setMinAssetsAmount` with no lower-bound enforcement.

The result: in a no-cooldown vault with `minAssetsAmount = 0`, a user can call `redeem(dustShares)`, burn their shares, and receive zero assets in return, with no revert and only a misleading `Unstaked(msg.sender, 0)` event.

Source: `issuance/src/vault/StakingVault.sol:297–340, 543–557`.

Impact: Dust share amounts can be silently burned for zero assets in the no-cooldown configuration. The behaviour is inconsistent with the cooldown path (which reverts on zero output) and inconsistent with the `ZeroAmount` error the contract otherwise uses to signal this condition. No direct attacker profit; the burned shares benefit remaining stakers marginally through increased share price.

Recommended Mitigation: Apply the zero-output check before returning in both no-cooldown branches:

```
if (_cooldownDuration == 0) {
    assets = _redeemTo(shares, msg.sender);
    if (assets == 0) revert ZeroAmount();
    return assets;
}
```

```
if ($.cooldownDuration == 0) {
    shares = _withdrawTo(assets, msg.sender);
    if (shares == 0) revert ZeroAmount();
    return shares;
}
```

```
}

```

Syntetika: Fixed in commit [5c0f783](#)

Cyfrin: Verified.

7.3.11 Fully-vested yield becomes unvested on vesting period update

Description: The `Distributor::updateTimelock` function allows the admin to update the vesting duration. However, the function neither blocks changes during an active vesting cycle nor does it reset `vestingAmount` after the cycle has ended.

If the admin updates the vesting period during an active cycle or after rewards have fully vested, the `StakingVault::getUnvestedAmount` function will recompute vesting using the new `vestingPeriod`, making already vested rewards becomes unvested again and reduce `StakingVault::totalAssets`.

```
uint256 deltaT;
unchecked {
    deltaT = (vestingPeriod - timeSinceLastDistribution);
}

return (deltaT * $.vestingAmount) / vestingPeriod;
```

Impact: The bug causes several economic issues for the vault:

1. Sudden share price crash: The ERC4626 share price is derived from `totalAssets`. When vested yield suddenly becomes unvested again, the asset value drops immediately.
2. Direct user loss: All vault share holders lose value. Users redeeming immediately after the update (due to transaction order in mempool) receive fewer assets because the contract effectively removes part of their already-earned yield.
3. Arbitrary opportunity: An attacker can withdraw at the high share price before the admin update, followed by a deposit at the deflated price.

Proof of Concept: Let's take an example:

- Assume initially, `vestingAmount` = 3600 and `vestingPeriod` = 3600s (1 hour).
- At $T = 3601s$, the vesting cycle is considered complete. Due to this, `getUnvestedAmount` returns 0 and `totalAssets` includes the 3600 tokens.
- Admin calls `updateTimelock(7200)` to extend the period from 1 hour to 2 hours.
- `getUnvestedAmount` calculates unvested amount using the old `vestingAmount` (3600) and new period (7200).
-

$$Unvested = \frac{(7200 - 3601) \times 3600}{7200} = 1799.5$$

- `totalAssets` instantly drops by ~1800 tokens.

Recommended Mitigation: Consider implemented the following logical changes in `Distributor::updateTimelock`:

```
/// @notice Updates the time lock duration.
function updateTimelock(
    uint256 _timeLock
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_timeLock > 0, InvalidTimeLock());

+   if (getUnvestedAmount() > 0) {
+       revert StillVesting();
+   }

+   if (vestingAmount > 0) vestingAmount = 0
```

```

DistributorStorage storage $ = _getDistributorStorage();
$.timeLock = _timeLock;
emit TimeLockUpdated(_timeLock);
}

```

Syntetika: Fixed in commit [dee5502](#)

Cyfrin: The fix added:

```

if (IStakingVault($.minter.stakingVault()).getUnvestedAmount() > 0) {
    revert StillVesting();
}

```

This correctly blocks updates **during** an active vesting cycle. But the recommended mitigation also asked for resetting vestingAmount to 0 when the cycle has already **fully** vested, and that part was not implemented.

The vulnerability still exists: if yield fully vests (timeSinceLastDistribution >= timeLock, so getUnvestedAmount() == 0), the check passes. Then if timeLock is increased to a larger value, timeSinceLastDistribution < newTimeLock, and the stale \$.vestingAmount in vault storage produces spurious unvested yield again.

Example from the original report: vestingAmount = 3600, old timeLock = 3600s. At T+3601 everything is vested, getUnvestedAmount() == 0. Admin updates timeLock = 7200. Immediately, getUnvestedAmount() returns ~1799 and share price drops. This window persists until the next distribution resets vestingAmount.

The active-vesting guard is now in place but the post-vesting stale state is not handled. The second line of the recommended mitigation, resetting vestingAmount to 0 in the vault before changing the timeLock, is still missing.

Syntetika: Fixed in commit [602c864](#)

Cyfrin: Verified. The fix adds a resetVesting() function and wires it into the updateTimeLock flow. The full call chain: Distributor::updateTimeLock - Minter::resetVesting() -> StakingVault::resetVesting() which sets \$.vestingAmount = 0.

7.3.12 Minter::distributeYield missing whenNotPaused

Description: Every externally-callable state-mutating function in Minter is gated by whenNotPaused except three that create synthetic supply:

Function	whenNotPaused
mint (L405)	✓
redeem (L419)	✓
transferToCustody (L392)	✓
realizeLosses (L432)	✓
distributeYield (L250)	MISSING
ownerMint (L232)	Admin
requestOwnerMint (L219)	Admin

Distributor::distributeYield is permissionless (rate-limited by timeLock). When Minter is paused, a bot or attacker can still call Distributor::distributeYield → Minter::distributeYield, minting new synthetic supply and inflating totalDeposits. If the pause was triggered in response to an oracle/feed compromise, the attacker-controlled feed value continues to propagate through the vault despite the emergency halt.

Impact: The pause-induced freeze of user-visible flows (mint, redeem) creates a false sense of containment. The two supply-creation paths that actually matter for an oracle or admin-key compromise continue operating. Emergency responders believe the system is frozen; it is not.

Recommended Mitigation: Add `whenNotPaused` to `Minter::distributeYield` (line 250). If yield-during-pause is intentional, document the asymmetry explicitly.

Syntetika: Fixed in commit [b58ab42](#)

Cyfrin: Verified.

7.3.13 `BlacklistableUpgradeable::blacklist(address(0))` bricks all mint and burn paths across `HilToken` and `StakingVault`

Description: `blacklist(address _account)` performs no parameter validation and silently writes `_blacklisted[address(0)] = true`. ERC-20 `_mint` calls `_update(address(0), to, amount)` and `_burn` calls `_update(from, address(0), amount)`. Both paths are blocked by the `notBlacklisted` guards on `from` and `to` respectively. After one `blacklist(0)` call:

- `Minter::mint` → `HilToken::mint` → `_update(address(0), to, amount)` → `reverts UserBlacklisted()`.
- `Minter::redeem` → `HilToken::burnFrom` → `_burn` → `_update(from, address(0), amount)` → `reverts`.
- `StakingVault::deposit` → `_mint` → `_update(address(0), shares_to, shares)` → `reverts`.
- `StakingVault::claimWithdraw` → `IHilToken.burn` → `_update(vault, address(0), amount)` → `reverts`.

Recommended Mitigation: Add `require(_account != address(0), AddressCantBeZero())` at entry of `BlacklistableUpgradeable::blacklist`.

Syntetika: Fixed in commit [cd54186](#). Protected addresses will be set during deployment, `address(0)` will be set as protected

Cyfrin: Verified.

7.4 Informational

7.4.1 StakingVault::_withdraw last-user branch reverts with ERC20InvalidReceiver when owner is renounced to zero - DoS of last-user exit during vesting window

Description: When a withdrawal makes `totalSupply() - shares == DEAD_SHARES` (last real user exiting) and unvested yield still exists, `_withdraw` sweeps the entire unvested amount to `owner()`:

```
if (totalSupply() - shares == DEAD_SHARES) {
    uint256 remainingUnvested = getUnvestedAmount();
    if (remainingUnvested > 0) {
        $.vestingAmount = 0;
        $.lastDistributionTimestamp = 0;
        IERC20(asset()).safeTransfer(owner(), remainingUnvested);
    }
}
```

StakingVault inherits OwnableUpgradeable via BlacklistableUpgradeable without overriding `renounceOwnership`. Neither function has a 2-step acceptance pattern (Ownable2StepUpgradeable exists in OZ but is not used). After an honest admin `renounceOwnership()` (plausible decentralization move), `owner() == address(0)`:

1. Distribution fires; `vestingAmount > 0`, `getUnvestedAmount() > 0`.
2. Last real staker attempts to exit: `redeem(allShares) -> _withdraw` hits the `totalSupply() - shares == DEAD_SHARES` branch -> `safeTransfer(owner()==0, remainingUnvested)` -> reverts with `ERC20InvalidReceiver(address(0))`.
3. Last staker DoSed until `getUnvestedAmount() == 0` (full `vestingPeriod` decay).

Sources: inherited OZ OwnableUpgradeable; StakingVault.sol:598-605.

Note: the generic `transferOwnership/renounceOwnership` inherited behavior is reported as a Centralization disclosure (intended OZ Ownable semantics). The subcase reported here is the honest-admin-triggered DoS path on `_withdraw`.

Impact: Permanent repeating DoS of last-user exit whenever unvested yield exists after an honest `renounceOwnership`. User must either wait for `getUnvestedAmount() == 0` (up to `vestingPeriod`) or leave ≥ 1 wei of shares behind (may conflict with `minAssetsAmount`).

Recommended Mitigation:

- Override `renounceOwnership()` to revert - the StakingVault first-deposit path and `_withdraw` last-user sweep depend on a live owner
- Guard the transfer: `if (remainingUnvested > 0 && owner() != address(0)) { safeTransfer(owner(), remainingUnvested); }` and leave `remainingUnvested` in the vault otherwise
- Migrate `BlacklistableUpgradeable` to `Ownable2StepUpgradeable` so ownership transfer requires explicit `acceptOwnership`
- Bind `owner()` strictly to `DEFAULT_ADMIN_ROLE`: override every `onlyOwner` call-site to require `onlyRole(DEFAULT_ADMIN_ROLE)` on the same caller, OR drop `onlyOwner` in favor of role-gated modifiers

Syntetika: Fixed in commit [536b5d3](#)

Cyfrin: Verified. `renounceOwnership` reverts.

7.4.2 TokensHolder::withdraw + Distributor::rescueERC20 use raw transfer without SafeERC20

Description: Both functions ignore transfer's return value. For tokens that return `false` on failure without reverting, failures are silent. `TokensHolder` is the sole funds-exit path for vault withdrawals - a silent failure wipes the cooldown record while leaving tokens stuck. `Distributor::rescueERC20` is admin-only but inconsistent with the `safeTransfer/safeTransferFrom` used elsewhere in the same files.

Sources: `issuance/src/helpers/TokensHolder.sol:34-36`; `issuance/src/vault/Distributor.sol:142-151`.

Recommended Mitigation: Use `SafeERC20.safeTransfer`. Add an event to `rescueERC20` for observability.

Syntetika: Fixed in commit [39a1cfd](#)

Cyfrin: Verified.

7.4.3 `StakingVault::setEarlyExitFeeRecipient` defaults to `address(this)` and permits self-target - fees silently boost share price

Description: Default fee recipient is the vault itself. Early-exit fees returned to the vault via `tokensHolder.withdraw($.earlyExitFeeRecipient, fee)` increase `balanceOf(vault)` and therefore `totalAssets()` instantly - bypassing the vesting mechanism and inflating share price for remaining holders. Absent admin action, all early-exit fees are ploughed back silently.

Source: `issuance/src/vault/StakingVault.sol:126, 175-182`.

Recommended Mitigation: Either document the self-recycling behaviour, or `require(newRecipient != address(this))` and initialize with a treasury address.

Syntetika: Fixed in commit [cdd5107](#)

Cyfrin: Verified.

7.4.4 `StakingVault::_deposit` with `minAssetsAmount > DEAD_SHARES` bricks vault seeding

Description: The first-deposit path burns `DEAD_SHARES = 1000` wei of assets. If `_minAssetsAmount > 1000`, the dead-shares mint reverts `AmountBelowLimit`. The vault cannot be initialized until admin calls `setMinAssetsAmount(<= 1000)` first. Unrecoverable without upgrade if admin fails to sequence correctly.

Source: `issuance/src/vault/StakingVault.sol:190-206, 610-622`.

Recommended Mitigation: In `initialize`, `require(_minAssetsAmount <= DEAD_SHARES, ConfigInvalid())`. Or special-case receiver == `BURN` in `_deposit` to skip the min check.

Syntetika: Fixed in commit [3face43](#)

Cyfrin: Verified.

7.4.5 `StakingVault::setDistributor` and `Minter::setDistributor` are independent - rotating one without the other silently bricks the yield pipeline

Description: Yield flow: `Distributor::distributeYield` -> `Minter::distributeYield` (requires `DISTRIBUTOR_ROLE` on Minter) -> `StakingVault::distributeYield` (requires `DISTRIBUTOR_ROLE` on Vault). Both Minter and StakingVault track the distributor address independently. Each `setDistributor` only rotates its own role binding. No atomic "replace distributor everywhere" helper. If admin rotates one side but not the other (or Minter's 2-step waits 1 day while Vault's is single-step), the yield pipeline reverts with role check failures.

Sources: `issuance/src/vault/StakingVault.sol:138-148`; `issuance/src/minter/Minter.sol:301-319`.

Recommended Mitigation: Document the co-requirement in NatSpec on both setters. Consider adding a governance helper that atomically drives both rotations (accepting that Minter's side is inherently 2-step after the Minter-side distributor-rotation fix).

Syntetika: Fixed in commit [ff6434d](#)

Cyfrin: Verified.

7.4.6 `CUSTODIAN_ROLE` is granted/revoked but never gates any function

Description: Minter declares `CUSTODIAN_ROLE` and manages it through `requestCustodianChange/setCustodian`, but no function uses `onlyRole(CUSTODIAN_ROLE)`. The role is dead code - the custodian is used only as the destination in `transferToCustody` (which is gated by `OPERATOR_ROLE`, not `CUSTODIAN_ROLE`). Readers will assume this role grants privileges it does not. Related drift risk: `transferToCustody` uses `$.custodian` storage, not the role

set, so the custodian address and role membership can diverge - off-chain tooling treating the role as meaningful is misled.

```
issuance/src/minter/Minter.sol
44:     bytes32 public constant CUSTODIAN_ROLE = keccak256("\CUSTODIAN_ROLE\");
```

Source: issuance/src/minter/Minter.sol:44, 198, 344-346.

Recommended Mitigation: Either:

- Remove the role entirely (and the grant/revoke bookkeeping), OR
- Gate the recipient of `transferToCustody` with `hasRole(CUSTODIAN_ROLE, recipient)` so the role has operational meaning, OR
- Add a function that actually requires `onlyRole(CUSTODIAN_ROLE)`.

Syntetika: Fixed in commit [27bcd8](#)

Cyfrin: Verified. CUSTODIAN_ROLE removed.

7.4.7 Distributor::rescueERC20 uses string error messages inconsistent with project style and ignores transfer return value

Description: Every other validation in the project uses custom errors, but `rescueERC20` uses string revert reasons. Also `IERC20(token).transfer(to, amount)` ignores the return value and does not use `SafeERC20`, unlike the rest of the contract.

```
issuance/src/vault/Distributor.sol
147:     require(token != address(0), "TokenRescuer: token is zero address");
148:     require(to != address(0), "TokenRescuer: recipient is zero address");
150:     IERC20(token).transfer(to, amount);
```

Recommended Mitigation:

```
require(token != address(0), AddressCantBeZero());
require(to != address(0), AddressCantBeZero());
IERC20(token).safeTransfer(to, amount);
```

Also emit an event for the rescue so it is observable off-chain.

Syntetika: Fixed in commit [4a233ae](#)

Cyfrin: Verified.

7.4.8 Missing NatSpec on public-facing functions with non-obvious behaviour

Description: Public/external functions with surprise-potential semantics lack documentation. In particular `HilToken::requestTransfer` has no NatSpec and no access control; `StakingVault::redeem` and `StakingVault::withdraw` ignore owner/receiver params - NatSpec does not mention this; `StakingVault::getEarlyExitAmount` percent/fee math is non-obvious.

Recommended Mitigation: Add NatSpec stating (a) that `redeem/withdraw` ignore the passed owner/receiver addresses and always operate on `msg.sender`, and (b) unit conventions for `getEarlyExitAmount` (BPS, linear interpolation, etc.).

Syntetika: Fixed in commit [6d1e8e0](#)

Cyfrin: Verified. `redeem/withdraw` use owner/receiver params as expected and `StakingVault::getEarlyExitAmount` percent/fee math is documented in natspec.

7.4.9 BlacklistableUpgradeable::_Blacklistable_init callable publicly during initialization

Description: The initializer is public onlyInitializing rather than internal onlyInitializing.

Recommended Mitigation: Change visibility to internal.

Syntetika: Fixed in commit [d7840b3](#)

Cyfrin: Verified.

7.4.10 TokensHolder::withdraw discards transfer return value and missing event

Description: Uses plain IERC20.transfer without checking return value and without SafeERC20. The contract has no event for withdrawals. Also STAKING_VAULT and HILBTC immutable variables lack the private visibility marker.

```
issuance/src/helpers/TokensHolder.sol
34:     function withdraw(address to, uint256 amount) external onlyStakingVault {
35:         HILBTC.transfer(to, amount);
36:     }
```

Recommended Mitigation: Use SafeERC20.safeTransfer, emit TokensWithdrawn(to, amount), and mark the immutables private (or expose them deliberately).

Syntetika: Fixed in commit [8662c20](#)

Cyfrin: Verified.

7.4.11 StakingVault::_withdraw resets \$.lastDistributionTimestamp to 0

Description: When the post-withdraw total supply falls to DEAD_SHARES, the code zeroes vestingAmount and lastDistributionTimestamp. Setting lastDistributionTimestamp = 0 is unusual - getUnvestedAmount later computes block.timestamp - lastDistributionTimestamp which will produce a huge number, but since vestingAmount = 0 the product is also 0. Relies on a non-obvious invariant.

Recommended Mitigation: Either drop the lastDistributionTimestamp = 0 write (only vestingAmount = 0 is needed to make getUnvestedAmount return 0) or document the invariant.

Syntetika: Fixed in commit [72caefc](#)

Cyfrin: Verified. lastDistributionTimestamp = 0 dropped.

7.4.12 Documentation mismatches: NatSpec, docstrings and comments disagree with code

Description: Grouping of documentation-versus-code mismatches across the codebase. Each sub-item is a distinct instance of NatSpec, a docstring, or an inline comment that disagrees with actual contract behavior.

1. HilToken::burnFrom NatSpec claims owner requires allowance

burnFrom skips _spendAllowance when spender == owner(). The NatSpec says "The sender must have allowance for the from address" - which is false when sender is owner.

```
issuance/src/token/HilToken.sol
161:     * @dev The sender must have allowance for the `from` address.
...
167:     function burnFrom(address from, uint256 amount) external {
168:         address spender = msg.sender;
169:         if (spender != from && spender != owner()) {
170:             _spendAllowance(from, spender, amount);
171:         }
172:         _burn(from, amount);
173:     }
```

Recommended: Update NatSpec to clarify the owner bypass, e.g. "...unless the sender is the owner, in which case allowance is bypassed." Owner burning arbitrary user balances is a privileged power that should ideally be emitted as a distinct event.

2. StakingVault::setMaxEarlyExitFeeBps **strict** < **makes documented 10% cap unreachable**

Comment says "(10%)"; MAX_FEE = 1_000 (10% in BPS). The check require(newValue < MAX_FEE, IncorrectEarlyExitFee()) rejects exactly 10%, so the advertised max is unreachable. Also inconsistent with setCooldownDuration which uses non-strict <= MAX_COOLDOWN_DURATION.

Source: issuance/src/vault/StakingVault.sol:163-170.

Recommended: Change to <= or correct the NatSpec/comment to say "<10%".

3. HilBTC and HilUSD contract-level NatSpec copied verbatim from HilToken

Both files declare the same @dev "ERC20 token with permit functionality, minting and burning, and role-based access control." NatSpec - they should describe their specialization (decimals difference, asset type).

Recommended: Update NatSpec to state decimals and intended base asset.

Syntetika: Fixed in commits [5411680](#), and [ab4c806](#)

Cyfrin: Verified.

7.4.13 Missing zero-address and zero-amount input validation across constructors, initializers and setters

Description: Grouping of missing non-zero / input-validation checks across constructors, initializers, and two-step setters. Each sub-item is a distinct instance.

1. TokensHolder **constructor lacks non-zero validation on stakingVault and hilbtc**

Constructor stores whatever is passed. With stakingVault == 0, the onlyStakingVault gate rejects all callers - cooldown funds permanently stuck. With hilbtc == 0, HILBTC.transfer(...) reverts on codeless call. Contract is non-upgradeable so a bad init is unrecoverable.

Source: issuance/src/helpers/TokensHolder.sol:19-22.

Recommended: require(stakingVault != address(0) && hilbtc != address(0), AddressCantBeZero()).

2. Minter::requestOwnerMint, Minter::initialize, StakingVault::initialize **missing non-zero checks**

Minter::initialize zero-checks every address except _distributor - a zero-init distributor combined with the setDistributor zero-skip branch allows the very first distributor assignment to skip the waiting period entirely. requestOwnerMint accepts any amount (including 0) and any to (including address(0) - causes a stale request to revert on execute, DoSing owner-mint until re-requested). StakingVault::initialize has the same _distributor != 0 omission: if passed zero, getUnvestedAmount() reverts on the external IDistributor(0).vestingPeriod() call, bricking totalAssets() and every ERC-4626 path until admin calls setDistributor.

Sources: issuance/src/minter/Minter.sol:137-188, 219-228.

Recommended: Add require(_distributor != address(0), AddressCantBeZero()) in BOTH Minter::initialize AND StakingVault::initialize. Add require(to != address(0) && amount > 0, InvalidRequest()) in requestOwnerMint.

3. Two-step setters missing `pendingChange.timestamp != 0` guard

Setters read `pendingChange.newAddr` and execute without guarding against `pendingChange.timestamp == 0`. A double-executed `set*` call reads `newAddr = 0` and `timestamp = 0`; `0 + waitingPeriod <= block.timestamp` is trivially true; roles are granted to `address(0)`, bricking future functionality until another request cycle.

Sources: `issuance/src/minter/Minter.sol:301-347, 362-375`; `issuance/src/token/HilToken.sol:126-141`.

Affects `Minter::setCustodian`, `Minter::setPauser`, `Minter::setDistributor`, `HilToken::setMinter`.

Recommended: Add `require($.pendingChange.timestamp != 0, NoPendingRequest())` to every executor.

Syntetika: Fixed in commit [256aa81](#), no check for distributor is intentional

Cyfrin: Verified.

7.4.14 Solidity compiler version and toolchain configuration drift across the repo

Description: Grouping of compiler-version and toolchain configuration inconsistencies across the repo.

1. Pragma drift across in-scope contracts

Different files pin different pragmas: `^0.8.26` for most files; `^0.8.20` for `TokensHolder`; `^0.8.19` for `StakingVault` and `Distributor`. Foundry configures `solc_version = "0.8.26"` so all files compile there, but the floor pragmas silently allow building against older versions that miss features used elsewhere.

Recommended: Align to `^0.8.26` across all in-scope files so static analyzers and downstream tooling consistently target the same version.

2. Hardhat vs Foundry version mismatch; Hardhat optimizer disabled

The Hardhat config compiles `deposit-registry` contracts with `0.8.28`, while the issuance Foundry config uses `0.8.26`. Hardhat config does NOT specify an optimizer - disabled by default, producing larger, more expensive bytecode.

Recommended: Enable the optimizer in `hardhat.config.ts` and align the Solidity version with `foundry.toml`:

```
solidity: {
  version: "0.8.26",
  settings: {
    optimizer: { enabled: true, runs: 200 },
    viaIR: true,
  },
},
```

Syntetika: Fixed in commit [564be0](#)

Cyfrin: Verified.

7.4.15 Cosmetic code cleanups: typo, unused imports, inconsistent types and wrong error name

Description: Grouping of trivial identifier/string cleanups across the codebase. All are non-behavioral and can be fixed mechanically.

1. Typo PreviousAmoutNotVestedYet - missing n

Custom error name misspells "Amount".

```
issuance/src/minter/Minter.sol
256:         PreviousAmoutNotVestedYet()
```

Renaming the error is a breaking change for ABIs.

Recommended: Rename to PreviousAmountNotVestedYet.

2. HilToken::setMinter reverts with wrong error name WaitingPeriodBelowMinimum

When the cooldown for a minter change has not elapsed, setMinter reverts with WaitingPeriodBelowMinimum - the same error used by updateWaitingPeriod when the new period is below the minimum. The semantically correct error is WaitingPeriodNotElapsed (used identically in Minter and for the UUPS upgrade path in HilToken itself).

```
issuance/src/token/HilToken.sol
128:         if ($.minter != address(0)) {
129:             require(
130:                 $.pendingMinterChange.timestamp + $.requiredWaitingPeriod <=
131:                 block.timestamp,
132:                 WaitingPeriodBelowMinimum()
133:             );
134:         }
```

Recommended: Replace WaitingPeriodBelowMinimum() with WaitingPeriodNotElapsed().

3. Unused imports in Minter

IERC4626 in Minter.sol line 21 is not referenced anywhere in the file.

Recommended: Remove the unused imports.

Syntetika: Fixed in commit [3d11f17](#)

Cyfrin: Verified.

7.4.16 Inconsistent access-control patterns: mixed Ownable/AccessControl and mixed internal/external role revocation

Description: Grouping of stylistic/consistency issues in how access control is implemented. Neither is an exploitable gap, but both are drift risks and sources of reader confusion.

1. StakingVault mixes onlyOwner with onlyRole(DEFAULT_ADMIN_ROLE)

StakingVault mixes two access-control models - most admin setters use onlyRole(DEFAULT_ADMIN_ROLE) but setCooldownDuration and setMinAssetsAmount use onlyOwner. Since Ownable's owner is set via __Blacklistable_init(_initialAdmin), these happen to converge on the same address at init, but the inconsistency is a drift risk: future admin-role changes via AccessControl will not affect owner and vice versa.

Recommended: Pick one access-control model (AccessControl roles) for the whole contract.

2. Minter mixes _revokeRole with external revokeRole

Within the same contract, `setCustodian` calls `revokeRole(CUSTODIAN_ROLE, $.custodian)` (public, re-runs access control), while `setDistributor` and `setPauser` call `_revokeRole(...)` (internal). Both paths are gated by `DEFAULT_ADMIN_ROLE` so both succeed, but the style is inconsistent and the public version wastes gas on duplicate access-control checks.

Recommended: Use `_revokeRole` consistently.

Syntetika: Fixed in commit [9b6a03e](#)

Cyfrin: Verified.

7.4.17 `HilToken::_update` and `StakingVault::_update` check blacklist status of `address(0)`

Description: Both `_update` overrides check blacklist status of every address involved in a token movement, `from`, `to`, and `msg.sender`, without first excluding the ERC-20 sentinel address(0).

ERC20::`_mint` calls `_update(address(0), to, amount)`: `from` is `address(0)`. ERC20::`_burn` calls `_update(from, address(0), amount)`: `to` is `address(0)`.

HilToken (non-owner path, `issuance/src/token/HilToken.sol:212-217`):

```
require(
    !_isBlacklisted(from) &&    // + address(0) checked during _mint
    !_isBlacklisted(to) &&      // + address(0) checked during _burn
    !_isBlacklisted(msg.sender),
    UserBlacklisted()
);
```

StakingVault (`issuance/src/vault/StakingVault.sol:575-577`):

```
notBlacklisted(from)    // + address(0) checked during share mint (deposit)
notBlacklisted(to)      // + address(0) checked during share burn (withdraw)
notBlacklisted(msg.sender)
```

`address(0)` is not a real participant — it holds no tokens, has no signing key, and cannot be a counterparty in any compliance context. Checking it against the blacklist is semantically meaningless and creates an avoidable attack surface: any mechanism that writes `_blacklisted[address(0)] = true` (whether via the `blacklist()` function directly, a storage collision, or an upgrade bug) silently bricks every mint and burn across both contracts simultaneously.

Mitigation:

Guard both `from` and `to` blacklist checks with a non-zero sentinel exclusion:

HilToken:

```
if (msg.sender != owner()) {
    require(
        (from == address(0) || !_isBlacklisted(from)) &&
        (to == address(0) || !_isBlacklisted(to)) &&
        !_isBlacklisted(msg.sender),
        UserBlacklisted()
    );
}
```

StakingVault:

```
function _update(address from, address to, uint256 amount)
    internal override
    notBlacklisted(msg.sender)
{
    if (from != address(0)) _requireNotBlacklisted(from);
    if (to != address(0)) _requireNotBlacklisted(to);
}
```

```
    super._update(from, to, amount);  
}
```

This aligns with standard compliance-token practice (USDC, EUROCC) where `address(0)` is implicitly excluded from blacklist checks because it participates only as an ERC-20 accounting sentinel, never as a real token holder.

Syntetika: Fixed in commit [6ced19](#)

Cyfrin: Verified.

7.5 Gas Optimization

7.5.1 Cache repeated storage reads on success path across multiple setters

Description: Several functions re-read the same storage slot multiple times on the success path. Caching in a local saves ~100 gas per subsequent warm SLOAD:

- `Minter::setDistributor` (\$.distributor read on initial check then again in `_revokeRole`)
- `HilToken::setMinter` (\$.minter read twice)
- `StakingVault::redeem` (\$.cooldowns[msg.sender] written 3 times)
- `StakingVault::withdraw` (\$.cooldowns[msg.sender] accessed 4x, \$.cooldownDuration read twice)
- `StakingVault::claimWithdraw` (\$.tokensHolder read up to 3 times)

Recommended Mitigation: Cache the storage slot or storage pointer into a local (e.g., `address _distributor = $.distributor;`, `UserCooldown storage userCooldown = $.cooldowns[msg.sender];`) and reuse it.

Syntetika: Fixed in commit [340b912](#)

Cyfrin: Verified.

7.5.2 Use `calldata` instead of `memory` for external function string parameters

Description: Several external/public functions take `string memory` parameters that are only read, not modified. Changing to `calldata` avoids an unnecessary memory copy.

```
issuance/src/vault/StakingVault.sol
211:     string memory referral
240:     string memory referral
254:     string memory referral
267:     string memory referral
```

Recommended Mitigation: For external `stake` and external `mint(shares, referral)`, change `string memory referral` to `string calldata referral`. The public `deposit/mint` overloads called internally through `stake` must remain `memory` - verify call graph before applying.

Syntetika: Fixed in commit [ac01ad9](#)

Cyfrin: Verified.